

Model order reduction for a transient problem using the Laplace transform

Keyvan Attarian^{1,3} and Amine Ammar^{2,3}

¹Ecole polytechnique, Institut Polytechnique de Paris

²LAMPA Laboratory & ESI Group Chair, Arts et Metiers Institute of Technology

³DesCartes Program, WP8, CNRS@CREATE

Printed July 16, 2026

Abstract

Model order reduction for parametric transient partial differential equations (PDEs) typically suffers from the high computational cost of predicting long time horizons. To alleviate this, we propose a surrogate modeling strategy that leverages the Laplace transform to map the time domain into a highly compressed set of complex frequency fields. We systematically compare two non-linear compression approaches using autoencoders: Spatial Laplace Autoencoders (SLAE), which compress individual frequency frames, and Latent Laplace Autoencoders (LLAE), which apply the transform directly within a low-dimensional latent space. Our results demonstrate that end-to-end training of the surrogate models significantly improves prediction accuracy. The best surrogate model (LLAE, $d_z = 128$, $K = 16$) achieves a median relative L^2 error of 4.13%, a 54% reduction compared to the best linear POD surrogate (8.91%), while running in under 20 ms per sample on GPU.

Contents

1	Introduction	2
2	Methodology	3
3	Results	16
4	Comparison and Discussion	33
	Appendices	36

1 Introduction

High-fidelity simulations of parametric transient partial differential equations (PDEs), such as those governing combustion, reactive transport, and atmospheric dispersion, rely on computationally expensive numerical solvers. In many-query contexts like optimization, uncertainty quantification, real-time control, and digital twins, the complete spatiotemporal field $U(x, t; \mu)$ must be evaluated for thousands of parameter values μ [7, 22]. The prohibitive cost of these repeated evaluations motivates the development of surrogate models capable of reproducing the full dynamics in milliseconds. Applications such as air quality and pollution forecasting concretely illustrate this need: global physical models are accurate but unusable in real-time without approximation [11, 4].

Classical model order reduction (MOR) methods [13], such as Reduced Basis (RB) and Proper Orthogonal Decomposition (POD) combined with interpolation, construct a low-dimensional linear subspace by projecting high-fidelity solutions onto spatial modes. These approaches are theoretically well-founded and highly effective for smooth dynamics. However, they suffer from a **reconstruction floor** linked to the linearity of the basis: for problems with sharp transients, localized structures, or strongly non-linear dynamics (such as convection-diffusion-reaction equations), the number of required modes explodes. The transition toward **non-linear manifolds** becomes necessary as linear techniques struggle [5].

Beyond the spatial compression bottleneck, classical MOR methods also struggle with the temporal dimension. The treatment of time is either sequential (time-stepping on the projected ROM) or space-time (variational), both of which impose a computational cost that grows with the temporal horizon [8, 17, 26]. This dual bottleneck — spatial and temporal — motivates the approach developed in this paper. The **parameter-to-field** setting — predicting a full spatiotemporal field from a low-dimensional parameter vector $\mu \in \mathbb{R}^p$ — is addressed by **DL-ROMs** (Deep Learning Reduced-Order Models) [18, 5, 6]: a convolutional autoencoder compresses the spatial field into a compact latent code, a surrogate network learns $\mu \mapsto z$, and the decoder reconstructs the full field at inference, yielding significant gains over linear ROMs. This is a different problem from **field-to-field** operator learning (FNO [19], DeepONet [20, 21]), which takes an input function rather than a scalar parameter vector; the two settings are not directly comparable. Parameter-conditioned U-Nets [15] represent a lighter DL-ROM variant for 2D problems, operating directly in the time domain. However, all existing DL-ROM approaches share a common temporal bottleneck: they retain the full temporal resolution of the trajectory, one latent vector per time step. Whether the surrogate takes time as an input coordinate and is queried once per step, $g(\mu, t) \mapsto z(t)$ [5], or propagates the latent sequentially in time, often autoregressively [25] at the cost of error accumulation, the temporal work — the number of network evaluations, or the output size — scales with N_t . For long transients, this makes the learning problem harder and inference heavier, regardless of how compact the spatial latent is.

A natural path to address this bottleneck is to work in the **frequency domain**: compressing the time axis into $K \ll N_t$ spectral coefficients reduces the surrogate output dimension dramatically. The spectral intuition is well-established in the neural operator literature — FNO [19] parameterizes its kernel in Fourier space — but the Fourier transform is poorly suited for transient signals: it assumes periodicity and exhibits spectral leakage for non-periodic dynamics. The **Laplace transform** $s = \gamma + i\omega$ generalizes Fourier by adding a real decay axis γ that naturally encodes transients, exponential stability, and non-periodic regimes. Each Laplace frequency field $\hat{U}(s_k)$ is a static spatial

field, structurally simpler to compress than a full temporal sequence. The Laplace Neural Operator (LNO) [2] confirms this advantage in the field-to-field setting. On the linear reduction side, Reyes [28] and Goeckner et al. [9] show that Laplace-domain POD bases are exponentially accurate for parametric evolution problems — yet these approaches remain fundamentally linear. The remaining gap is to combine the Laplace spectral compression with the non-linear expressivity of autoencoders within the DL-ROM framework, enabling **end-to-end** differentiable training thanks to the exact invertibility of the transform [10].

This paper fills that gap. We situate our work in the DL-ROM family and introduce the Laplace transform as a **temporal** compression stage on top of the standard AE-surrogate pipeline: instead of predicting N_t latent frames, the surrogate predicts $K \ll N_t$ spectral coefficients, from which the full trajectory is recovered by inversion. Here $U(t) \in \mathbb{R}^{N_x}$ is the discretized spatial field at time step t , N_x the number of spatial nodes, N_t the number of time steps, and K the number of Laplace frequencies. Two instantiations are studied depending on where the transform is applied — in the physical domain (SLAE) or in the latent space (LLAE) — and compared against a linear POD baseline. Our contributions are as follows:

- **Temporal compression via the Laplace transform (main contribution):** we propose and evaluate using the Laplace transform to compress the temporal dimension of the surrogate target from N_t frames to $K \ll N_t$ spectral coefficients, making prediction cost independent of the temporal horizon.
- **SLAE vs. LLAE:** we systematically compare two instantiations of the Laplace compression — in the physical domain (SLAE) and in the latent space (LLAE) — and characterize the accuracy/latency trade-off they offer.
- **Optimal contour selection:** for LLAE, the Laplace poles are learned end-to-end alongside the surrogate; for SLAE, we derive a bias-variance decomposition of the reconstruction error and optimize the contour offline by minimizing a principled proxy loss that trades regularization bias against surrogate approximation error.

The remainder of this paper is organized as follows. Section 2 describes the methodology, detailing the Laplace transform, surrogate architectures, and the end-to-end training pipeline. Section 3 presents the numerical results and performance comparisons. Finally, Section 4 provides a discussion and concludes the work. The appendices detail the governing equations and architectural hyperparameters.

2 Methodology

2.1 Laplace Transform and Inversion

Throughout this section, $u(t) \in \mathbb{R}$ denotes the field value at a single fixed spatial location (lowercase = one pixel); for the full spatial field $U(t) \in \mathbb{R}^{N_x}$ (uppercase = full grid of N_x nodes), all operations below are applied independently at each grid point. The forward Laplace transform is evaluated at discrete frequencies $s = \gamma + i\omega$ using a temporal quadrature rule (e.g., trapezoidal rule with weights w_n):

$$(1) \quad \hat{u}(s) = \int_0^T u(t) e^{-st} dt \approx \Delta t \sum_{n=0}^{N_t-1} w_n u(t_n) e^{-st_n}$$

Since $u(t)$ is strictly real-valued, its Laplace transform satisfies the conjugate symmetry property $\hat{u}(\bar{s}) = \overline{\hat{u}(s)}$. Consequently, we restrict our numerical evaluation to frequencies with non-negative imaginary parts $\omega_k \geq 0$, which halves the forward evaluation cost while providing the full spectral information required for inversion.

Regularized Inversion (Tikhonov). Let $\mathbf{F} \in \mathbb{C}^{K \times N_t}$ be the forward evaluation matrix, with $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$, and let $\hat{\mathbf{u}} \in \mathbb{C}^K$, $\mathbf{u} \in \mathbb{R}^{N_t}$ be the vectors of Laplace coefficients and temporal values. Taking advantage of the real-valued constraint on $\mathbf{u}$, the reconstruction is formulated as a regularized complex least-squares problem:

$$(2) \quad \mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^{N_t}} \|\mathbf{F}\mathbf{u} - \hat{\mathbf{u}}\|_{\mathbb{C}^K}^2 + \alpha_t \|\mathbf{D}_t \mathbf{u}\|_2^2 + \lambda \|\mathbf{u}\|_2^2$$

where $\mathbf{D}_t \in \mathbb{R}^{(N_t-1) \times N_t}$ is the first-difference operator penalizing temporal roughness, and $\lambda > 0$ is a Tikhonov ridge regularization parameter. The corresponding optimality condition (normal equations) yields a real symmetric linear system:

$$(3) \quad \left(\Re(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I} \right) \mathbf{u}^* = \Re(\mathbf{F}^H \hat{\mathbf{u}})$$

By solving this system, the conjugate frequencies are implicitly accounted for through the real-part operator $\Re(\cdot)$, ensuring a real-valued temporal reconstruction. Since the inversion is implemented via a linear solve (e.g., `torch.linalg.solve`), the operator is fully differentiable and compatible with gradient-based training. Crucially, this formulation holds for any set of frequencies (s_k), with no constraint on their distribution in the complex plane.

Choice of contour: truncated Bromwich. A key design choice is the set of evaluation frequencies ($s_1, \dots, s_K$). We adopt throughout the **truncated Bromwich contour**: a vertical line in the complex plane at $\Re(s) = \gamma$, with imaginary parts equal to the first K discrete Fourier frequencies,

$$(4) \quad s_k = \gamma + i \frac{2\pi k}{N_t \Delta t}, \quad k = 0, \dots, K-1,$$

where $\gamma > 0$ is the **damping parameter** and $K \leq \lfloor N_t/2 \rfloor + 1$ controls the spectral truncation. With this choice, the forward sum (1) reduces to a truncated DFT of the exponentially damped sequence:

$$(5) \quad \hat{u}(s_k) = \text{FFT}\{u(t_n) \Delta t w_n e^{-\gamma t_n}\}_k, \quad k = 0, \dots, K-1,$$

making the forward pass computationally efficient. Since $K < \lfloor N_t/2 \rfloor + 1$, the spectrum is incomplete and we rely on the Tikhonov inversion (2)–(3) in all cases. Restricting to the Bromwich structure, however, imposes that all poles share the same real part γ and lie on a regular frequency grid — a strong constraint that may be suboptimal. Since the Tikhonov inversion handles any set of frequencies, one can in principle place the poles freely in the complex plane to better capture the signal’s dynamics. This is the motivation for Section 2.4, which discusses how to optimize the contour beyond the Bromwich family.

2.2 Autoencoders in the Laplace Domain

We investigate two autoencoder architectures that differ solely in **where** the Laplace transform is applied, as illustrated in Figure 1. In the **Spatial Laplace AE (SLAE)**, the

transform is applied first in the physical domain, and a frequency-conditioned autoencoder then compresses each of the K resulting complex frequency frames independently. In the **Latent Laplace AE (LLAE)**, the order is reversed: a time-conditioned autoencoder first encodes the full temporal sequence into a compact latent trajectory, and the Laplace transform is subsequently applied within that low-dimensional space.

In both cases, an L_2 ridge penalty on the latent representations enforces a sufficiently regular structure, which makes the latent space easier for the surrogate to navigate (Section 2.3).

Both architectures build on the same convolutional encoder–decoder backbone, applied directly to the spatial field with no POD pre-reduction; this is the standard deep convolutional-autoencoder compression of the DL-ROM lineage [18, 5], as opposed to the POD-then-autoencoder pre-reduction of POD-DL-ROM [6]. On top of this shared backbone, both architectures use the same conditioning mechanism: a sinusoidal coordinate encoding combined with Feature-wise Linear Modulation (FiLM) [27]. The coordinate $\tau \in [0, 1]$ represents either the frequency ratio $f_k = k/(K - 1)$ for SLAE, or the normalised time step t/T for LLAE.

Sinusoidal Coordinate Encoding. The coordinate $\tau \in [0, 1]$ is embedded via sinusoidal basis functions inspired by NeRF positional encoding [23]:

$$(6) \quad \mathbf{e}(\tau) = \text{MLP}\left(\left[\sin(2^i \pi \tau), \cos(2^i \pi \tau)\right]_{i=0}^{L-1}\right), \quad L = 8.$$

A plain scalar τ would conflate nearby coordinates and fail to capture sharp transitions in either the frequency or time domains. The multi-scale sinusoidal basis explicitly represents τ at L different resolutions, providing the network with a rich, non-saturating signal that allows fine discrimination across the entire range.

FiLM Conditioning. To share encoder/decoder weights across all coordinates (frequencies or time steps) while adapting to each specific configuration, the FiLM mechanism is employed. At each convolutional layer ℓ , the coordinate embedding $\mathbf{e}(\tau)$ is projected to a channel-wise gain $\alpha_\ell \in \mathbb{R}^{C_\ell}$ and bias $\delta_\ell \in \mathbb{R}^{C_\ell}$, modulating the activations element-wise:

$$(7) \quad \mathbf{x}_\ell \leftarrow (\mathbf{x}_\ell \odot (1 + \alpha_\ell(\mathbf{e}(\tau)))) + \delta_\ell(\mathbf{e}(\tau)).$$

The residual formulation $(1 + \alpha_\ell)$ initializes the modulation close to the identity, which stabilizes early training. All coordinates share the same convolutional weights, and the FiLM parameters act as a lightweight per-coordinate adapter at each layer, enabling a single autoencoder to process the full spectrum or temporal sequence without training separate model copies.

2.2.1 Spatial Laplace AE (SLAE)

SLAE first applies the Laplace transform at the pixel level, converting the N_t -frame temporal sequence $U(t)$ into K complex-valued spatial fields $\hat{U}(s_k)$. Each frequency frame, represented as a two-channel (real/imaginary) image, is independently compressed by a FiLM-conditioned convolutional encoder into a real latent vector $z(s_k) \in \mathbb{R}^{d_z}$:

$$(8) \quad U(t) \xrightarrow{\mathcal{L}_{\text{pixel-wise}}} \hat{U}(s_k) \xrightarrow{\text{Encoder}(f_k)} z(s_k) \xrightarrow{\text{Decoder}(f_k)} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

The decoder reconstructs $\hat{U}(s_k)$ from $z(s_k)$ via separate output heads for the real and imaginary components. Training minimizes the Laplace-domain reconstruction error together with an L_2 regularization term:

$$(9) \quad \mathcal{L}_{\text{SLAE}} = \|\tilde{U}(s_k) - \hat{U}(s_k)\|_{\mathbb{C}^{N_x}}^2 + \beta \|z(s_k)\|_2^2$$

where $\|\cdot\|_{\mathbb{C}^{N_x}}^2$ is the squared L_2 norm over the N_x spatial nodes (real and imaginary parts combined), and β is a regularization hyperparameter.

2.2.2 Latent Laplace AE (LLAE)

LLAE encodes the physical frames $U(t)$ into a latent trajectory $z(t) \in \mathbb{R}^{d_z}$ using a time-conditioned convolutional encoder (conditioning on the normalised ratio t/T), then applies the Laplace transform directly within this compact latent space:

$$(10) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\mathcal{L}} \hat{z}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

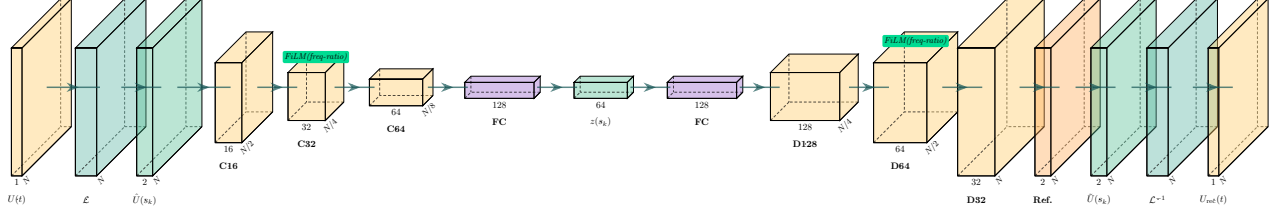
Compressing in the latent space rather than the physical domain offers two advantages: the transform operates on d_z -dimensional vectors instead of N_x -dimensional fields, and the encoder produces a smoother temporal representation than the raw complex frequency frames, easing the spatial compression task. The main counterpart is computational: at each training step and at inference, the encoder and decoder must process all N_t physical frames, whereas SLAE processes only the K frequency frames. Since $N_t \gg K$, LLAE is significantly heavier, which limits the network capacity that can be deployed under a fixed compute budget. The loss combines three terms:

$$(11) \quad \begin{aligned} \mathcal{L}_{\text{LLAE}} = & \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 \\ & + \beta_{\text{latent}} \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|\tilde{z}(t_n) - z(t_n)\|_2^2 \\ & + \beta \frac{1}{K} \sum_{k=0}^{K-1} \|\hat{z}(s_k)\|_{\mathbb{C}^{d_z}}^2 \end{aligned}$$

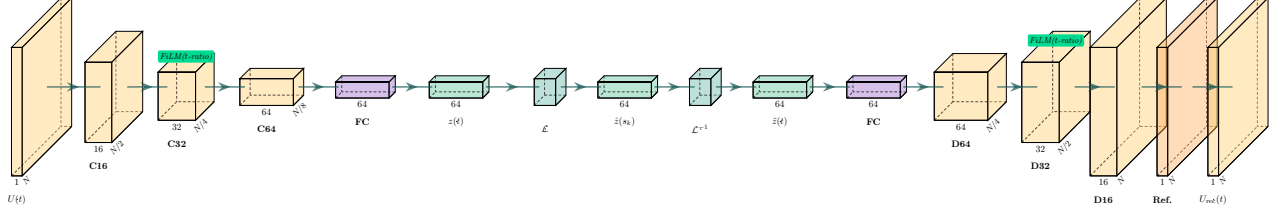
which combines the physical-time reconstruction error, a latent consistency term penalizing the round-trip error $\tilde{z}(t_n) - z(t_n)$, and an L_2 regularization penalty on the Laplace-domain latent codes $\hat{z}(s_k)$. The latent consistency term is unique to LLAE: it explicitly enforces the strict invertibility of the continuous Laplace transform within the learned discrete latent space, a structural constraint not required by SLAE since it encodes each frequency frame independently.

2.3 Surrogate Training

Following the DL-ROM paradigm of [5, 6], the surrogate MLP g_θ (shared trunk + per-frequency heads; architecture detailed in Figure 2 and Appendix E) replaces the encoding steps by learning the direct mapping $\mu \mapsto w_k$, where w_k denotes the Laplace-domain latent at frequency s_k that feeds the decoder: $w_k = z(s_k) \in \mathbb{R}^{d_z}$ for SLAE, and $w_k = \hat{z}(s_k) \in \mathbb{C}^{d_z}$ for LLAE. Two strategies for constructing the surrogate target w_k are described below.



(a) SLAE: pixel-wise Laplace transform, then per-frequency convolutional AE ($d_z = 64$, $K = 16$).



(b) LLAE: convolutional AE first, then Laplace transform in latent space ($d_z = 64$, $K = 16$).

Figure 1: SLAE (a) and LLAE (b) autoencoder pipelines.

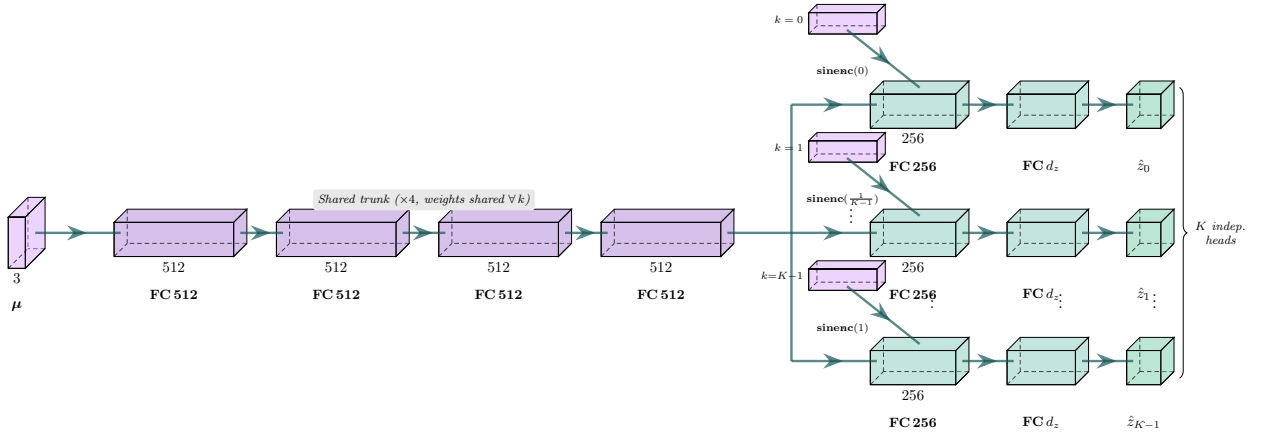


Figure 2: **Surrogate MLP architecture.** A shared trunk maps physical parameters μ to a latent representation, which is broadcast to K independent heads. Each head is conditioned on a frequency index via sinusoidal encoding and predicts the target latent w_k for that frequency.

2.3.1 Direct Embedding Prediction

In the direct approach, the surrogate predicts the full latent vector at each index without any preliminary compression. For SLAE, the encoder produces frequency-domain latents $z(s_k) \in \mathbb{R}^{d_z}$ directly, and the pipeline is:

$$(12) \quad \mu \xrightarrow{g_\theta} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

For LLAE, the time-domain latent sequence $z(t) \in \mathbb{R}^{d_z}$ is first transformed to the Laplace domain, and the surrogate predicts those Laplace-domain codes directly:

$$(13) \quad \mu \xrightarrow{g_\theta} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

2.3.2 Latent Sequence SVD Compression

To reduce the effective output dimension of the surrogate and simplify its learning task, one can apply a preliminary offline SVD compression to the Laplace-domain latents before training the surrogate, an approach inspired by [24]. For both SLAE and LLAE, the SVD is applied in the frequency domain: given the K Laplace-domain latent codes $\{\hat{z}(s_k)\}_{k=1}^K$ collected over the training set, a truncated SVD yields a projection matrix $V \in \mathbb{R}^{d_z \times k_{\text{svd}}}$ (with $k_{\text{svd}} \ll d_z$) such that $\hat{z}(s_k) \approx (\hat{z}(s_k)V)V^\top$. The reduced codes $\hat{G}(s_k) = \hat{z}(s_k)V \in \mathbb{R}^{k_{\text{svd}}}$ become the surrogate targets, reducing the prediction problem from dimension d_z to k_{svd} .

In the **SLAE-SVD** variant (latents indexed by frequency k): the projection yields $G(s_k) = z(s_k)V$, and the surrogate predicts in the compressed frequency-domain space:

$$(14) \quad \mu \xrightarrow{g_\theta} G_{\text{pred}}(s_k) \xrightarrow{V^\top} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

In the **LLAE-SVD** variant, the SVD is likewise applied in the Laplace domain: the projection yields $\hat{G}(s_k) = \hat{z}(s_k)V$, and the surrogate predicts the compressed frequency-domain latents before back-projection and inversion:

$$(15) \quad \mu \xrightarrow{g_\theta} \hat{G}_{\text{pred}}(s_k) \xrightarrow{V^\top} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

Because V is real and acts on the latent channel index while $\mathcal{L}^{-1}$ acts on the frequency index, the two commute: back-projecting before or after the inversion is equivalent, and the implementation applies whichever order is cheaper.

In both strategies, the entire pipeline is trained end-to-end to minimise the total physical-time loss:

$$(16) \quad \mathcal{L}_{\text{total}} = \alpha_{\text{lat}} \|w_k^{\text{pred}} - w_k^{\text{true}}\|^2 + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|^2$$

where w_k^{pred} and w_k^{true} denote the predicted and true Laplace-domain latents (either $z(s_k)$ or $\hat{G}(s_k)$ depending on the chosen strategy). End-to-end backpropagation allows gradients from the spatiotemporal MSE to flow through the decoder and the inverse transform, jointly updating the surrogate weights, the AE decoder, and, in the SVD variant, the projection basis V .

2.3.3 Tucker Decomposition in Latent Space

Just like the SVD approach described in the previous section, the goal here is to further compress the latent representations of both LLAE and SLAE before training the surrogate MLP, which leads to two distinct variants: **LLAE-Tucker** and **SLAE-Tucker**. While the SVD approach reduces the latent sequence along a single mode at a time (for LLAE-SVD it collapses the latent dimension d_z ; for SLAE-SVD it collapses d_z independently at each frequency, ignoring cross-frequency correlations), Tucker decomposition [16] generalises this by simultaneously factorising two modes of the latent tensor.

Crucially, for both variants, the Tucker decomposition is applied in the latent space **after** the Laplace transform. We assemble the frequency-domain latents of the training set into a three-way tensor $\hat{\mathcal{Z}} \in \mathbb{C}^{N_{\text{train}} \times K \times d_z}$ (real-valued for SLAE, complex-valued for LLAE). A Tucker- (r_μ, r_s, r_z) decomposition approximates each scalar element of the tensor as a triple sum:

$$(17) \quad \hat{\mathcal{Z}}_{i,j,m} \approx \sum_{p=1}^{r_\mu} \sum_{l=1}^{r_s} \sum_{q=1}^{r_z} \hat{C}_{p,l,q} U_{i,p}^{(\mu)} U_{j,l}^{(s)} U_{m,q}^{(z)},$$

where $\hat{C} \in \mathbb{C}^{r_\mu \times r_s \times r_z}$ is the global core tensor, and $U^{(\mu)}, U^{(s)}, U^{(z)}$ are the orthogonal factor matrices for the parameter (sample), frequency, and latent modes, respectively. To enable predictions on unseen test samples, we absorb the parameter-dependent terms into a simulation-specific core matrix $\hat{\mathcal{G}}_i \in \mathbb{C}^{r_s \times r_z}$, whose elements are $(\hat{\mathcal{G}}_i)_{l,q} = \sum_{p=1}^{r_\mu} \hat{C}_{p,l,q} U_{i,p}^{(\mu)}$. This allows us to rewrite the approximation for the full frequency-domain latent matrix $\hat{\mathcal{Z}}_i \in \mathbb{C}^{K \times d_z}$ of the i -th simulation in a compact matrix form:

$$(18) \quad \hat{\mathcal{Z}}_i \approx U^{(s)} \hat{\mathcal{G}}_i U^{(z)\top}, \quad U^{(s)} \in \mathbb{C}^{K \times r_s}, \quad U^{(z)} \in \mathbb{C}^{d_z \times r_z}.$$

The shared orthogonal factors $U^{(s)}$ and $U^{(z)}$ are computed offline by the Higher-Order Orthogonal Iteration (HOOI) algorithm [3] and kept frozen. The surrogate then learns the mapping $\mu \mapsto \hat{\mathcal{G}}(\mu) \in \mathbb{C}^{r_s \times r_z}$, whose dimensionality $r_s \cdot r_z$ is substantially smaller than $K \cdot d_z$ because it jointly exploits cross-frequency and latent correlations.

For LLAE-Tucker, the surrogate predicts the complex-valued core matrix, from which the full Laplace-domain latents are reconstructed and inverted back to the time domain before decoding:

$$(19) \quad \mu \xrightarrow{g_\theta} \hat{\mathcal{G}}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} \hat{\mathcal{Z}}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{\mathcal{Z}}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

For SLAE-Tucker, the surrogate predicts the real-valued core matrix, which is expanded to the real-valued frequency latents, decoded to spatial Laplace fields, and finally inverted to the physical time domain:

$$(20) \quad \mu \xrightarrow{g_\theta} \mathcal{G}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} \mathcal{Z}_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

Unlike the real basis V of the SVD variant, the frequency factor $U^{(s)}$ mixes the Laplace frequencies and $U^{(z)}$ is complex-valued; neither commutes with $\mathcal{L}^{-1}$, so the back-projection necessarily precedes the inversion. Both variants share the same conceptual advantage: instead of predicting K independent vectors of size d_z , the surrogate predicts a highly compact core matrix that captures the essential spatiotemporal dynamics.

2.4 Choosing an Optimal Laplace Contour

The question of how to choose the integration contour $(s_1, \dots, s_K)$ arises for both SLAE and LLAE, but the two architectures admit fundamentally different strategies.

2.4.1 Learnable Contour for LLAE

In the LLAE setting, the Laplace transform is applied in the low-dimensional latent space: the surrogate directly predicts latent Laplace coefficients, and the complex frequencies s_k can be made learnable parameters of the model, optimized end-to-end with the rest of the network. This is possible because both the forward quadrature (1) — a finite sum of complex exponentials $e^{-s_k t_n}$, smooth in s_k — and the Tikhonov inversion (3) — a linear solve whose coefficient matrix A depends smoothly on s_k , α_t , and λ — are fully differentiable with respect to the contour and regularization parameters.

This is precisely what is done in our LLAE implementation: both the real parts $\sigma_k = \text{Re}(s_k)$ and imaginary parts $\omega_k = \text{Im}(s_k)$ of each of the K poles are learned independently, along with the regularization hyperparameters α_t and λ (via their log-parameterisation). The poles are initialised on the truncated Bromwich contour (ω_k at the K first FFT frequencies, $\sigma_k = \gamma_{\text{init}}$ uniform).

2.4.2 Offline Contour Optimisation for SLAE

In contrast, making the SLAE contour learnable would require recomputing all N_t spatial forward passes per gradient step on the frequencies — negating its primary computational advantage over LLAE. Instead, we seek an **offline** procedure to select a good fixed contour before training SLAE.

We present a theoretical framework for this purpose. The goal is to decompose the total reconstruction error $\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})$ into a **bias** term (introduced by regularization and the finite contour) and a **variance** term (the surrogate error propagated through the inversion), and to jointly optimize the contour and regularization parameters by minimising an upper bound on the expected reconstruction error.

Formal Setup. Using the notation of Section 2.1, we fix a spatial location and work with scalar (per-pixel) quantities. Let $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^{N_t}$ be the true temporal signal $(u(t_n))_n$ at that location, $\hat{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^K$ its exact Laplace coefficients $(\hat{u}(s_k))_k$, and $\tilde{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^K$ the surrogate-predicted coefficients (the per-pixel counterpart of the full-field $\tilde{U}(s_k)$ of Section 2.2.1). The regularised reconstructions from the true and predicted coefficients are, respectively,

$$(21) \quad \mathbf{u}^*(\boldsymbol{\mu}) = A^{-1} \Re \left(F^H \hat{\mathbf{u}}(\boldsymbol{\mu}) \right), \quad \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = A^{-1} \Re \left(F^H \tilde{\mathbf{u}}(\boldsymbol{\mu}) \right),$$

where $A = \Re(F^H F) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$ is the normal-equation matrix of (3) and $L_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$. The unique existence of these solutions is guaranteed since $\lambda > 0$ and $\mathbf{D}_t^T \mathbf{D}_t$ is positive semi-definite, making $L_{\text{reg}} \succeq \lambda \mathbf{I}$ and hence A symmetric positive definite (see Appendix F).

Bias–Variance Decomposition. We decompose the total reconstruction error by introducing $\mathbf{u}^*(\mu)$ and adding and subtracting it:

$$(22) \quad \begin{aligned} \mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu) &= (\mathbf{u}(\mu) - \mathbf{u}^*(\mu)) + (\mathbf{u}^*(\mu) - \mathbf{u}_{\text{rec}}(\mu)) \\ &= (\mathbf{u}(\mu) - A^{-1}\Re(F^H F)\mathbf{u}(\mu)) + A^{-1}\Re(F^H(\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu))). \end{aligned}$$

Using $A = \Re(F^H F) + L_{\text{reg}}$, the bias term simplifies to:

$$(23) \quad \mathbf{u}(\mu) - \mathbf{u}^*(\mu) = A^{-1}(A - \Re(F^H F))\mathbf{u}(\mu) = A^{-1}L_{\text{reg}}\mathbf{u}(\mu).$$

Taking norms in (22), applying the triangle inequality, and using $\|\Re(F^H \cdot)\| \leq \|F^H\| \|\cdot\|$ yields:

Theorem 2.1 (Bias–variance bound). *For any μ , the reconstruction error satisfies*

$$(24) \quad \|\mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu)\| \leq \underbrace{\|A^{-1}L_{\text{reg}}\mathbf{u}(\mu)\|}_{\text{regularization bias}} + \underbrace{\|A^{-1}F^H\|}_{\text{amplification factor}} \cdot \underbrace{\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^K}}_{\text{surrogate error}}.$$

The regularization bias depends on the true field $\mathbf{u}(\mu)$ and L_{reg} , and is mitigated by the well-posedness of A^{-1} . Any surrogate prediction error $\hat{\mathbf{u}} - \tilde{\mathbf{u}}$ is magnified during reconstruction by the amplification factor $\|A^{-1}F^H\|$, which plays the role of a condition number for the inversion process.

Optimization Problem Formulation. The matrices F and L_{reg} , and consequently A , depend explicitly on the contour $(s_1, \dots, s_K)$, on λ , and on α_t . We aim to jointly optimize these parameters to minimise the expected mean reconstruction error $\mathbb{E}_\mu[\|\mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu)\|]$. Since evaluating this true error directly is challenging without ground truth for every candidate contour, we treat the hyperparameters as learnable parameters and optimize a proxy loss $\mathcal{L}^{\text{proxy}}$ derived directly from the bound of Theorem 2.1:

$$(25) \quad \mathcal{L}^{\text{proxy}} = \mathcal{L}_{\text{bias}} + \mathcal{L}_{\text{var}}, \quad \mathcal{L}_{\text{bias}} = \mathbb{E}_\mu[\|\mathbf{u}(\mu) - \mathbf{u}^*(\mu)\|], \quad \mathcal{L}_{\text{var}} = \|A^{-1}F^H\| \cdot \epsilon_{\text{surr}},$$

where $\epsilon_{\text{surr}} = \mathbb{E}_\mu[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^K}]$ is the expected surrogate error. By Jensen’s inequality ($\mathbb{E}[\sqrt{X}] \leq \sqrt{\mathbb{E}[X]}$), this expected norm is bounded by:

$$(26) \quad \begin{aligned} \epsilon_{\text{surr}} &\leq \sqrt{\mathbb{E}_\mu[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^K}^2]} \\ &= \sqrt{\sum_{k=1}^K \mathbb{E}_\mu[|\hat{u}_k(\mu) - \tilde{u}_k(\mu)|^2]}, \end{aligned}$$

Approximating the Surrogate Error via SVD Residual. Training a full neural surrogate for every proposed contour is computationally intractable. We therefore approximate the per-frequency expected surrogate error using the mean SVD residual. Here $\hat{U}(s_k; \mu) \in \mathbb{C}^{N_x}$ denotes the **full spatial** Laplace field at frequency s_k (the vector of per-pixel values $\hat{u}_k(\mu)$ across all N_x locations), as opposed to the scalar $\hat{u}_k(\mu)$ at a single location used above. For any specific frequency s_k , let $U^{(k)} = [\hat{U}(s_k; \mu_1), \dots, \hat{U}(s_k; \mu_{N_{\text{train}}})] \in \mathbb{C}^{N_x \times N_{\text{train}}}$ be the dataset matrix formed by stacking the spatial Laplace fields across all

training instances. The expected squared error of the optimal linear rank- d_z surrogate equals the mean squared column-wise error:

$$(27) \quad \mathbb{E}_{\mu} [\|\hat{U}(s_k; \mu) - \hat{U}^{\text{lin}}(s_k; \mu)\|^2] = \frac{1}{N_{\text{train}}} \left\| \mathbf{U}^{(k)} - \hat{\mathbf{U}}_{\text{lin}}^{(k)} \right\|_F^2.$$

The minimum of this squared Frobenius norm over all rank- d_z approximations is achieved by the truncated SVD and equals exactly the sum of the discarded squared singular values. The optimal linear baseline error is therefore:

$$(28) \quad \begin{aligned} \mathcal{E}_{\text{SVD}}(s_k) &= \sqrt{\min_{\text{rank } \hat{\mathbf{U}} \leq d_z} \frac{1}{N_{\text{train}}} \left\| \mathbf{U}^{(k)} - \hat{\mathbf{U}} \right\|_F^2} \\ &= \sqrt{\frac{1}{N_{\text{train}}} \sum_{j > d_z} \left(\sigma_j^{(k)} \right)^2}, \end{aligned}$$

where $\sigma_j^{(k)}$ are the singular values of $\mathbf{U}^{(k)}$. This establishes the fundamental lower bound for the expected error of any linear surrogate of latent dimension d_z . Since a non-linear AE with bottleneck d_z generalizes the linear SVD, the optimal AE error satisfies $\epsilon_{\text{AE}}(s_k) \leq \mathcal{E}_{\text{SVD}}(s_k)$ (see Appendix F). In practice, the actual AE error may differ slightly due to optimisation challenges (local minima, architectural constraints), but for contour optimisation we are primarily interested in the **relative** behavior of ϵ_{surr} with respect to the contour rather than its absolute value. The SVD residual is fast and deterministic to compute for any candidate contour, making it highly suitable for iterative optimisation. To account for the enhanced expressivity of the non-linear AE over the linear SVD baseline, we introduce a scaling hyperparameter $\lambda_{\text{AE}} \geq 1$. The final proxy objective is:

$$(29) \quad \begin{aligned} \mathcal{L}^{\text{proxy}} &= \frac{1}{N_{\text{train}}} \sum_{\mu} \|\mathbf{u}(\mu) - \mathbf{u}^*(\mu)\| \\ &\quad + \frac{\|\mathbf{A}^{-1} \mathbf{F}^H\|}{\lambda_{\text{AE}}} \cdot \sqrt{\frac{1}{N_{\text{train}}} \sum_{k=1}^K \sum_{j > d_z} \left(\sigma_j^{(k)} \right)^2}. \end{aligned}$$

Minimizing (29) by gradient descent jointly optimizes the contour frequencies and the regularization parameters, trading bias against variance in a principled and fully differentiable manner.

2.5 Baselines

The approach described so far combines two ingredients: a non-linear autoencoder for spatial compression, and a Laplace transform for temporal compression. We therefore compare against two references, each removing exactly one of them. The first is **linear** in the spatial compression step but keeps the Laplace domain: a per-frequency POD basis with a non-intrusive surrogate on the projection coefficients (Section 2.5.1). The second is **non-linear** but Laplace-free, and keeps the full temporal grid: a standard DL-ROM (Section 2.5.2). Read together, they attribute the observed performance to one ingredient or the other rather than to their combination. Both reuse the surrogate of Section 2.3 and the two-phase training protocol unchanged.

2.5.1 Linear Baseline: POD in the Laplace Domain

As a linear reference, we construct a Proper Orthogonal Decomposition (POD) basis for each Laplace frequency independently. For each s_k , the training-set spatial fields are stacked into a data matrix $\mathbf{U}^{(k)} \in \mathbb{R}^{N_x \times N_{\text{train}}}$ (one per physical component), and a truncated basis of rank k_{pod} is extracted. Because $N_x = N^2 = 16,384$ and $N_{\text{train}} = 6,480$, a direct SVD of $\mathbf{U}^{(k)}$ would be memory-intensive; we instead use a two-pass randomized SVD [12] that reduces the cost to two passes over the data and a small k_{pod} -dimensional eigenvalue problem. The real and imaginary parts of $\hat{U}(s_k)$ are treated as separate real-valued fields, each with its own basis $\mathbf{V}^{(k,r)}, \mathbf{V}^{(k,i)} \in \mathbb{R}^{N_x \times k_{\text{pod}}}$ fitted on the training set:

$$(30) \quad \Re(\hat{U}(s_k)) \approx \mathbf{V}^{(k,r)} \mathbf{c}^{(k,r)}, \quad \Im(\hat{U}(s_k)) \approx \mathbf{V}^{(k,i)} \mathbf{c}^{(k,i)},$$

where $\mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} \in \mathbb{R}^{k_{\text{pod}}}$ are the projection coefficients. This is the classical Galerkin-POD representation applied frequency-by-frequency in the Laplace domain [13].

Surrogate for the POD coefficients. Given the compact coefficient vectors, the surrogate task reduces to predicting $2K k_{\text{pod}}$ scalar targets from $\boldsymbol{\mu}$. This is the classical non-intrusive POD-NN strategy [14], here applied frequency-by-frequency in the Laplace domain. The surrogate MLP g_θ of Section 2.3 predicts the normalized coefficients for all frequencies jointly:

$$(31) \quad \mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} = g_\theta(\boldsymbol{\mu}, k), \quad k = 0, \dots, K-1.$$

Training minimizes a combined latent-coefficient MSE and physical reconstruction loss (after Tikhonov inversion):

$$(32) \quad \begin{aligned} \mathcal{L}_{\text{POD-surr}} = & \alpha_{\text{lat}} \sum_{k=0}^{K-1} \left(\left\| g_\theta^{(k,r)}(\boldsymbol{\mu}) - \bar{\mathbf{c}}^{(k,r)} \right\|_2^2 + \left\| g_\theta^{(k,i)}(\boldsymbol{\mu}) - \bar{\mathbf{c}}^{(k,i)} \right\|_2^2 \right) \\ & + \alpha_{\text{spat}} \left\| U_{\text{rec}} - U \right\|_2^2, \end{aligned}$$

where $\bar{\mathbf{c}}^{(k,\cdot)}$ denotes the normalized coefficients and U_{rec} is the field reconstructed by back-projecting the predicted coefficients through the fixed bases and applying the inverse Laplace transform. The spatial bases $\mathbf{V}^{(k,\cdot)}$ remain **fixed** throughout surrogate training.

The time series $U(t)$ is reconstructed by back-projecting the predicted coefficients, assembling the complex frequency field, and applying the Tikhonov inversion (2). This pipeline is fully linear in the spatial compression step, making it a clean reference that quantifies the gain of non-linear autoencoders. The irreducible truncation error of the linear basis — controlled by the discarded singular values of $\mathbf{U}^{(k)}$ — sets a hard floor that non-linear AEs are designed to overcome.

2.5.2 Nonlinear Baseline: DL-ROM without Laplace

The POD baseline removes the non-linearity; our second reference removes the Laplace transform instead. It is a standard DL-ROM [5]: the convolutional encoder maps each physical frame to a latent code and the decoder reconstructs it, with no spectral compression anywhere in the pipeline:

$$(33) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

This is exactly the LLAE pipeline of Section 2.2.2 with the latent round trip $z(t) \rightarrow \hat{z}(s_k) \rightarrow \tilde{z}(t)$ removed: the same convolutional backbone, the same FiLM conditioning on the normalised time t/T , the same L_2 ridge on the latent codes. The loss retains the first and third terms of (11),

$$(34) \quad \mathcal{L}_{\text{DL-ROM}} = \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 + \beta \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|z(t_n)\|_2^2,$$

the ridge now applying to $z(t)$ directly rather than to $\hat{z}(s_k)$. The latent consistency term has no counterpart here: with no round trip, there is no invertibility to enforce. The comparison is therefore a controlled ablation of the Laplace block rather than a comparison between two independently tuned models. We deliberately use the DL-ROM of [5] rather than the POD-DL-ROM variant [6], whose POD pre-reduction is an efficiency device that leaves the learned representation unchanged; Section 4.5 returns to this point.

Surrogate cost. The surrogate of Section 2.3 carries over unchanged except in **what** it must predict. Where the Laplace pipelines regress the K Laplace-domain latents w_k , the DL-ROM surrogate must regress the latent trajectory at every time step, $\mu \mapsto z(t_n)$ for $n = 0, \dots, N_t - 1$: the per-frequency heads become per-time-step heads, conditioned on $t_n/(N_t - 1)$ in place of f_k , and their number grows from K to N_t . With a shared trunk of N_{trunk} parameters and heads of N_{head} parameters each — N_{head} depending only on d_z and the head width, not on their number —

$$(35) \quad N_{\text{surr}} = N_{\text{trunk}} + K N_{\text{head}}, \quad K = N_t \text{ (DL-ROM)}, \quad K \ll N_t \text{ (SLAE, LLAE)},$$

so the DL-ROM surrogate grows linearly with the temporal resolution, while the Laplace surrogates are independent of it. With the architecture of Appendix E and $d_z = 64$, this amounts to 23.5M parameters at $N_t = 150$ against 3.5M for LLAE at $K = 16$ — a factor of 6.8 — the two becoming equal in size at $N_t \approx 18$.

Temporal-resolution sweep. Since N_t is the sole driver of this cost, we sweep it by uniform subsampling of the time grid with a stride κ , so that $N_t \mapsto \lceil N_t/\kappa \rceil$ and $\Delta t \mapsto \kappa \Delta t$ while the horizon T is held fixed. This traces the DL-ROM cost/accuracy curve while the Laplace pipelines remain at $K = 16$, and it supplies a parameter-matched operating point: at $\kappa = 10$ the DL-ROM surrogate has essentially the size of LLAE at $K = 16$, both compressing time by roughly a factor of ten — uniformly on one side, spectrally on the other. The sweep concerns the DL-ROM alone; the Laplace pipelines are always trained on the full $N_t = 150$ grid, since their temporal compression is already carried by K .

2.6 Architecture Summary

Table 1 provides a synthetic overview of all nine pipelines studied. The two top rows describe the **autoencoder reconstruction** (Phase 1, input $U(t)$); the seven remaining rows compare the **surrogate inference** pipelines (Phase 2, input μ), grouped by the domain in which the Laplace transform is applied.

Model	Pipeline	$\mathcal{L}$ applied in
<i>Autoencoder reconstruction</i> (Phase 1, input: $U(t)$)		
SLAE		phys. domain
LLAE		latent space
<i>Surrogate inference</i> (Phase 2, input: μ)		
POD		phys. domain
SLAE-Direct		phys. domain
SLAE-SVD		phys. domain
SLAE-Tucker		phys. domain
LLAE-Direct		latent space
LLAE-SVD		latent space
LLAE-Tucker		latent space

Table 1: Architecture summary of the nine pipelines studied. **Top block** (Phase 1): autoencoder reconstruction, input $U(t)$. **Bottom block** (Phase 2): surrogate inference pipelines mapping $\mu \rightarrow U(t)$, grouped by the domain in which $\mathcal{L}$ is applied physical domain (SLAE family) or latent space (LLAE family). Colour code: ■ MLP surrogate; ■ Laplace $\mathcal{L}$ / inverse $\mathcal{L}^{-1}$; ■ SVD back-projection; ■ Tucker back-projection; ■ convolutional encoder E (wide→narrow) / decoder D (narrow→wide); • latent-space representation on the arrow.

3 Results

3.1 Problem Setting

We consider the emulation of the methane mass fraction Y in a two-dimensional parametric transient reacting flow. The transport equation is formulated as a convection-diffusion-reaction problem over a spatial domain $\Omega \subset \mathbb{R}^2$ and a time interval $t \in [0, T]$:

$$(36) \quad \frac{\partial(\rho Y)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y) - \nabla \cdot (\mu_{\text{eff}} \nabla Y) = \dot{\omega}_{\text{eff}}(Y; \boldsymbol{\mu})$$

where ρ is the fluid density, $\mathbf{U}$ is the velocity field, μ_{eff} is the effective diffusivity, and $\dot{\omega}_{\text{eff}}$ is the effective reaction rate. A detailed formulation of the complete system of governing equations is provided in Appendix D. The system is governed by a vector of three physical parameters $\boldsymbol{\mu} = [k, \theta, C]^\top \in \mathbb{R}^3$, specifically representing:

- k : Diffusion parameter,
- θ : Injection angle,
- C : Injection rate (kg/m³/s).

The spatial domain is discretized into a uniform grid of $N_x = 128 \times 128 = 16,384$ nodes, and the temporal domain into $N_t = 150$ time steps with uniform step size $\Delta t = 1$ s (total duration $T = 150$ s). The parameter space has dimension $p = 3$ ($\boldsymbol{\mu} \in \mathbb{R}^3$). The quantity of interest in the methodology notation is $U(t) \in \mathbb{R}^{N_x}$, the discretized spatial field of Y at time step $t \in \{1, \dots, N_t\}$; the full space-time field is a matrix in $\mathbb{R}^{N_x \times N_t}$. The number of Laplace frequency samples K is a model hyperparameter ($K = 16$ for both SLAE and LLAE; see Table 9). The dataset comprises 6480 training simulations and 1620 test simulations.

The performance of the models is evaluated quantitatively using the relative L^2 error over the held-out test set, defined as:

$$(37) \quad \epsilon_{\text{rel}} = \frac{\|Y_{\text{true}} - Y_{\text{pred}}\|_2}{\|Y_{\text{true}}\|_2}$$

We first characterize the autoencoder architectures in isolation, then evaluate the end-to-end surrogate models.

3.2 POD Baseline Results

We characterize the linear POD baseline (Section 2.5.1) by sweeping its three hyperparameters independently around a base configuration ($K = 16$, $k_{\text{pod}} = 64$, $\gamma = 0.01$): the number of Laplace frequencies $K \in \{8, 16, 32\}$, the number of POD modes $k_{\text{pod}} \in \{32, 64, 128\}$, and the damping parameter $\gamma \in \{0.0, 0.001, 0.01\}$. Each configuration is evaluated twice on the 1620 held-out test simulations: as a pure POD **reconstruction** (projecting the true test fields onto the training-set bases, no surrogate), which sets the linear error floor, and as the full end-to-end **surrogate**, where a network predicts the POD coefficients from $\boldsymbol{\mu}$ and the field is recovered by back-projection through the fixed bases and Tikhonov inversion. Table 2 reports both, and Figure 3 compares them side by side.

Linear reconstruction floor. The POD reconstruction error is governed by the singular value decay of the Laplace-domain field matrices $U^{(k)}$: the linear basis is accurate at

k_{pod}	K	γ	Rec med	Rec p90	Surr med	Surr p90	Gap (pp)	Lat (ms)
64	32	0.01	7.57	15.08	8.75	17.44	+1.17	5.6 ± 0.0
64	16	0.01	8.09	14.67	8.91	16.34	+0.82	3.5 ± 0.2
64	16	0.001	8.38	14.29	8.94	15.36	+0.56	3.5 ± 0.1
64	16	0.0	8.42	14.22	8.96	15.54	+0.54	3.5 ± 0.1
128	16	0.01	7.04	9.79	11.29	21.74	+4.26	3.7 ± 0.1
32	16	0.01	11.42	22.99	11.69	23.44	+0.27	3.4 ± 0.2
64	8	0.01	12.01	17.39	12.41	18.40	+0.41	2.9 ± 0.2

Table 2: POD baseline on the test set (1620 simulations), sorted by surrogate median error. All errors are relative L^2 errors in %. **Rec**: pure POD reconstruction error (linear floor, no surrogate). **Surr**: end-to-end surrogate error. **Gap**: difference surrogate – reconstruction (positive = surrogate adds error on top of the linear floor). **Lat**: GPU inference latency at batch size 1 (NVIDIA RTX A5000, 50 timed runs).

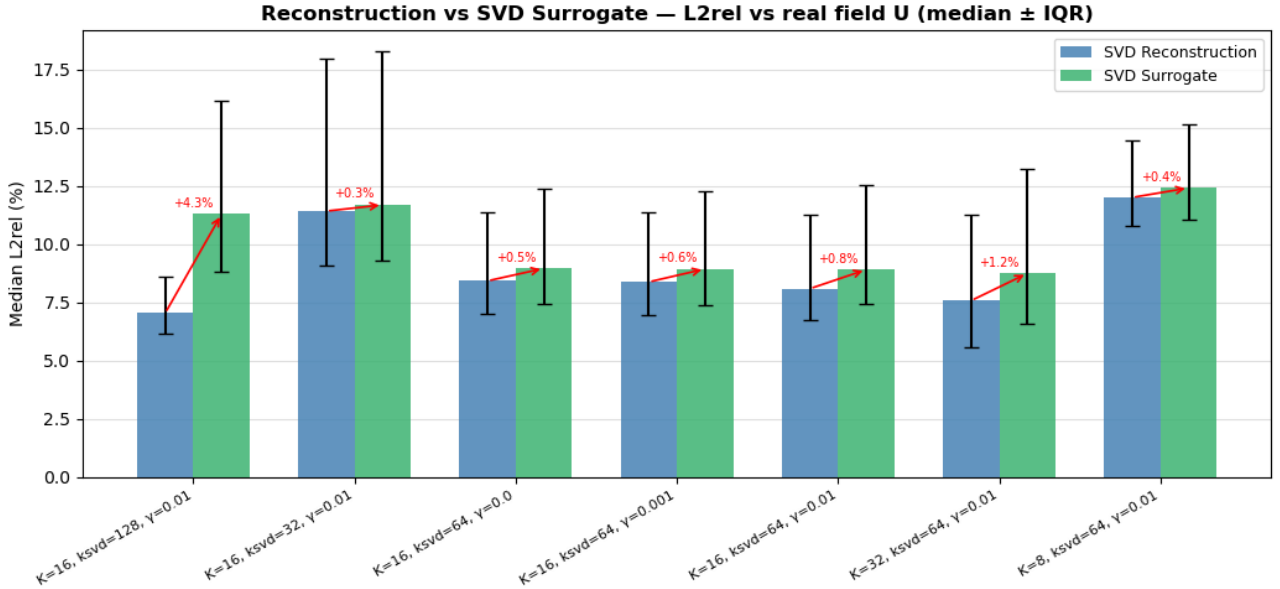


Figure 3: POD reconstruction (blue) vs. end-to-end surrogate (green): median relative L^2 error for all configurations (median $\pm$ IQR). Arrows indicate the surrogate overhead.

low Laplace frequencies, where the spatial mode structure is smooth and low-rank, but degrades steadily at higher frequencies where the fields become localized and the singular values decay more slowly. Increasing k_{pod} beyond 64 reduces this high-frequency error only marginally, indicating that the residual is not a truncation artifact but an intrinsic limit of the linear subspace; likewise, beyond $K = 16$ the dominant transient content is already captured and additional frequencies add noise rather than information. The damping γ is benign at the level of the global physical-field error (8.42%, 8.38%, 8.09% for $\gamma = 0, 0.001, 0.01$), because two error sources cancel: a purely imaginary contour concentrates energy more tightly and lowers the POD truncation error at every frequency, while a larger γ conversely damps the Tikhonov inversion error that propagates those residuals back to the time domain. Full ablations, including the per-frequency breakdown that exposes this trade-off, are given in Appendix B.

Surrogate overhead. For most configurations the surrogate step adds little above the linear floor: the gap ranges from +0.27 pp ($k_{\text{pod}} = 32$) to +1.17 pp ($K = 32$), confirming that the MLP predicts the POD coefficients from μ reliably. The exception is $k_{\text{pod}} = 128$, $K = 16$: despite the best reconstruction floor (7.04%), the surrogate error rises to 11.29% (gap +4.26 pp). With $2 \times K \times k_{\text{pod}} = 4,096$ scalar targets, the higher-dimensional regression problem overwhelms the benefit of the larger basis. The best POD surrogate is therefore $k_{\text{pod}} = 64$, $K = 16$, $\gamma = 0.01$ (8.91% median), the balance point between basis expressivity and surrogate learnability. Inference is fast throughout — 2.9 ms ($K = 8$) to 5.6 ms ($K = 32$) per sample at batch size 1, reaching 800–870 samples/s at batch size 32 — with the cost dominated by the back-projection through $V^{(k)}$ and the Tikhonov inversion rather than the network forward pass.

This establishes the linear POD as a well-characterized reference: its error floor, set by the discarded singular values of $U^{(k)}$, is the target that the non-linear autoencoders of Section 3.3 are designed to push below. Per-sample error distributions for all configurations are provided in Appendix A.

3.3 Autoencoder Performance

Table 3 reports the relative L^2 reconstruction error on the held-out test set (1620 simulations) for all trained AE checkpoints, evaluated on the physical field $U(t)$.

Figure 4 summarises the reconstruction floor across all 14 checkpoints. The two families present a stark contrast. LLAE achieves a best median error of 2.92% ($d_z = 128$, $K = 16$), a factor of four below the linear SVD baseline. SLAE yields errors in the range 7.78%–12.26%, broadly comparable to SVD: the **vs. SVD** column of Table 3 hovers around zero, positive as often as negative. Taken as a pure compression stage, SLAE does not offer a meaningful gain over the linear reference. This may appear disappointing, but the picture changes substantially at the surrogate stage (Section 3.4).

Two compounding factors explain this divergence (Sections 2.2.1–2.2.2). First, the input spaces differ: SLAE compresses high-dimensional complex frequency frames $\hat{U}(s_k)$, whereas LLAE encodes smoother physical frames — an intrinsically easier task. Second, the training objectives are asymmetric: LLAE’s loss (11) includes a direct physical-time reconstruction term that provides dense temporal supervision, while SLAE deliberately omits it during Phase 1 (Section 2.2.1) to preserve its computational advantage.

Figures 5 and 6 report ablation studies for each family, varying one hyperparameter at a time while holding the others fixed.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	vs. SVD (pp)
LLAE	128	16	0.01	2.92	3.15	4.38	-4.12
	64	32	0.01	3.19	3.53	5.11	-4.38
	64	16	0.01	3.20	3.47	5.04	-4.89
	64	16	0.0	3.26	3.61	5.24	-5.16
	64	16	0.001	3.32	3.65	5.16	-5.06
	64	8	0.01	3.35	3.66	5.26	-8.66
	32	16	0.01	3.43	3.85	5.60	-7.99
SLAE	64	16	0.0	7.78	8.11	9.91	-0.64
	64	16	0.001	7.82	8.27	10.36	-0.56
	128	16	0.01	7.98	8.40	10.45	+0.94
	64	32	0.01	8.34	9.27	13.43	+0.77
	64	16	0.01	8.92	9.53	12.55	+0.83
	32	16	0.01	10.76	11.14	14.76	-0.66
	64	8	0.01	12.26	12.74	15.40	+0.25

Table 3: Relative L^2 reconstruction error on the test set (1620 simulations). The last column gives the difference in median error relative to the linear POD baseline at matching $(K, k_{\text{pod}} = d_z, \gamma)$: negative values indicate improvement over POD, positive values indicate that POD outperforms the AE. Best per family in bold.

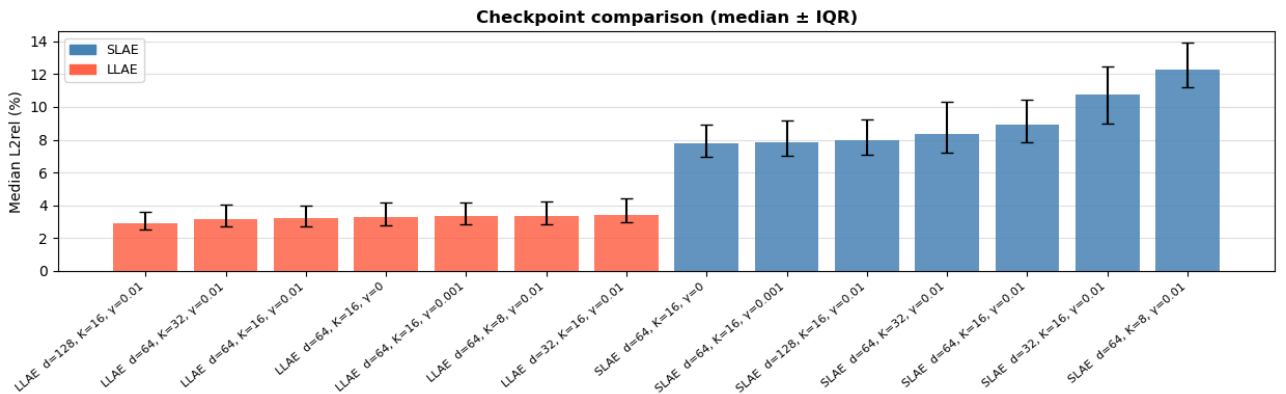


Figure 4: Reconstruction error for all 14 AE checkpoints, sorted by median relative L^2 error on the test set ($n = 1620$, error bars = IQR). LLAE in red, SLAE in blue.

Within LLAE (Figure 5), increasing the latent dimension from $d_z = 32$ to $d_z = 128$ reduces the median error from 3.43% to 2.92%, indicating that the latent bottleneck has not yet saturated at $d_z = 64$. The number of Laplace frequencies K has little impact: errors at $K = 8, 16$, and 32 are 3.35%, 3.20%, and 3.19% respectively, suggesting that 8–16 frequencies already capture the dominant transient dynamics. The damping parameter γ is similarly insensitive in the range $[0, 0.01]$, consistent with the smooth energy envelope of the exponential decay at these timescales.

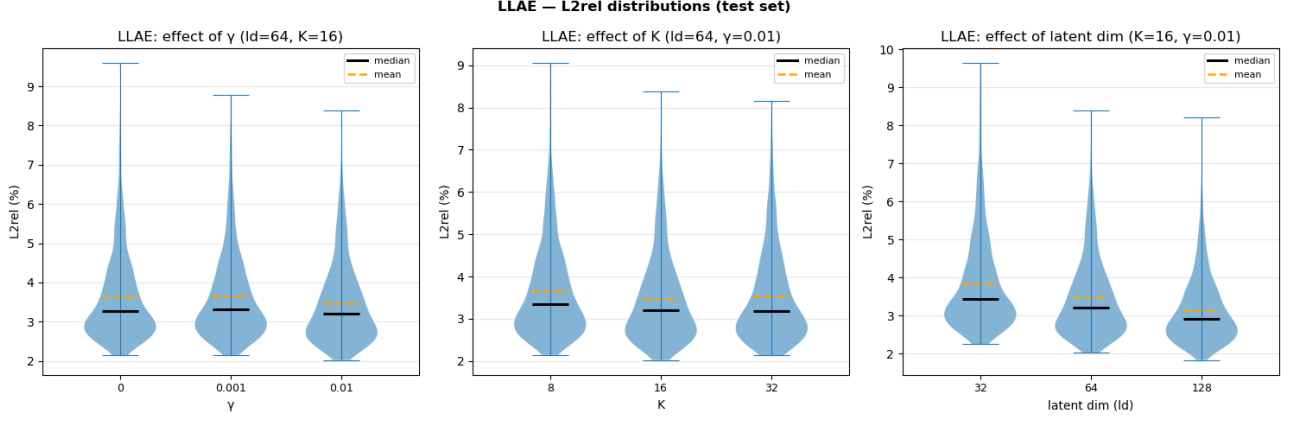


Figure 5: LLAE ablation study. From left to right: effect of latent dimension d_z ($K = 16$, $\gamma = 0.01$), number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$), and damping parameter γ ($d_z = 64$, $K = 16$). Violin plots show the full error distribution; black line is the median, dashed orange line is the mean.

Within SLAE (Figure 6), errors increase sharply at low K ($K = 8$: 12.26%) and low d_z ($d_z = 32$: 10.76%), pointing to a harder compression problem where both the spectral and spatial bottlenecks are binding simultaneously. Increasing K to 32 does not improve reconstruction (8.34% vs. 7.78% at $K = 16$), likely because higher Laplace frequencies carry less spatial energy and expose the spatial compression bottleneck. The SLAE error distribution also shows a heavier right tail (higher p90) than LLAE, reflecting occasional failure cases where spatial compression degrades on outlier samples.

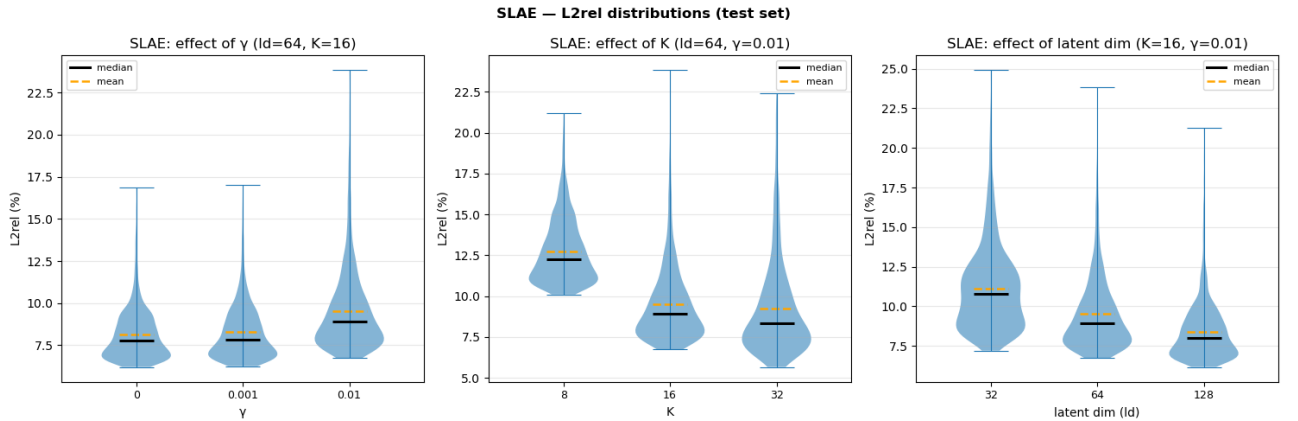


Figure 6: SLAE ablation study. Same layout as Figure 5 (fixed parameters: $K = 16$, $\gamma = 0.01$ / $d_z = 64$, $\gamma = 0.01$ / $d_z = 64$, $K = 16$). Note the larger y-axis range compared to LLAE.

For SLAE, Figure 7 shows the reconstruction error broken down per Laplace frequency k . The error is lowest at low frequencies ($k = 0-3$, where most signal energy is concen-

trated) and rises monotonically toward higher frequencies, confirming that the spatial compression bottleneck is more severe for the fine-scale, high-frequency content.

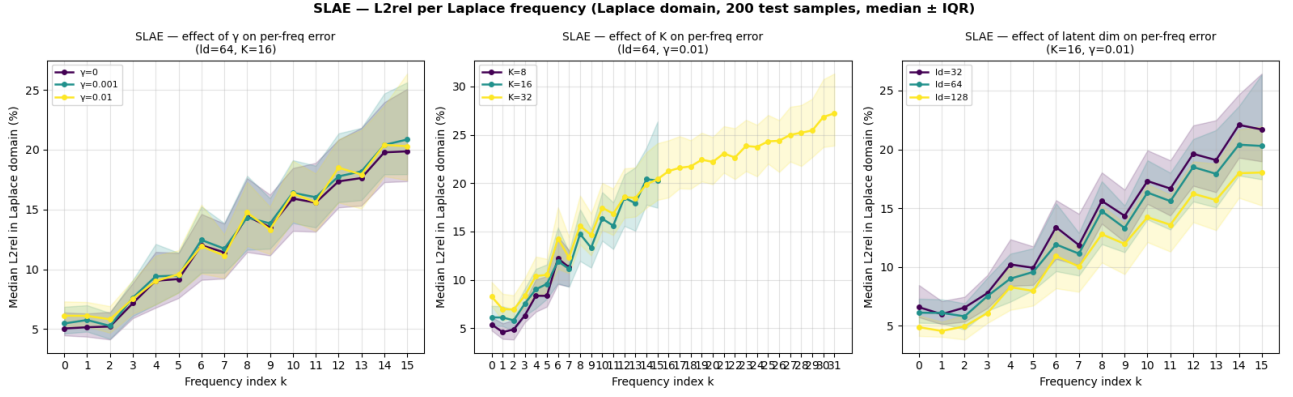


Figure 7: SLAE — per-frequency reconstruction error in the Laplace domain (median $\pm$ IQR, 200 test samples). Left: effect of γ ; centre: effect of K ; right: effect of d_z .

Comparing with the POD per-frequency breakdown (Figure 25), SLAE achieves roughly half the per-frequency error of POD at each k : the convolutional encoder captures non-linear structure that the linear projection cannot. Yet this per-frequency advantage does not translate into a better overall spatial reconstruction (Table 3). The reason is Tikhonov regularisation: the inverse transform (2)–(3) damps all frequencies uniformly, so the benefit of encoding each frequency more accurately is partially cancelled before the spatial field is recovered. The global spatial error is therefore a **regularisation-weighted** aggregate of the per-frequency errors, not a direct reflection of per-frequency compression quality.

Inference latency. Table 4 and Figure 8 report per-sample inference latency and GPU throughput measured with CUDA Events on an NVIDIA RTX 3090 (10 warm-up runs, 50 timed runs, batch size 1 for latency; max batch size for throughput). SLAE is approximately 2.4 $\times$ faster per sample than LLAE at $K = 16$ (8.0 ms vs. ~ 19 –20 ms), a direct consequence of the architectural difference detailed in Sections 2.2.1–2.2.2: SLAE decodes only K frequency frames at inference, whereas LLAE must decode all $N_t = 150$ time frames. At $K = 8$, SLAE latency drops to 5.5 ms. SLAE also requires far less GPU memory (112 MB at $K = 16$ vs. 525 MB for LLAE), enabling a 5 $\times$ larger maximum batch size.

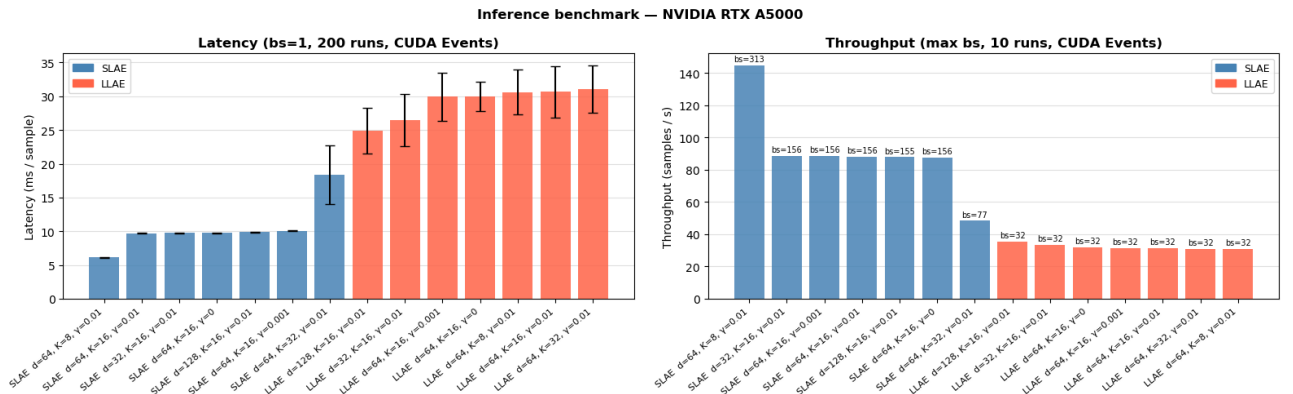


Figure 8: Inference benchmark on an NVIDIA RTX 3090. Left: per-sample latency (batch size 1, 50 CUDA Event runs). Right: throughput at maximum GPU-filling batch size.

Model	K	d_z	Latency (ms)	Throughput (s/s)	Peak GPU (MB)
SLAE	8	64	5.5 ± 0.2	224	56
SLAE	16	64	8.0 ± 0.1	134	112
SLAE	16	128	8.1 ± 0.1	133	112
SLAE	32	64	13.9 ± 0.4	74	224
LLAE	16	32	19.0 ± 1.1	56	525
LLAE	16	64	19.4–20.1	52–54	525
LLAE	16	128	19.0 ± 0.8	56	525

Table 4: Inference benchmark on an NVIDIA RTX 3090. Latency: batch size 1, 50 timed CUDA Event runs after 10 warm-up runs. Throughput at maximum GPU-filling batch size.

3.4 Surrogate Prediction Performance

3.4.1 Direct surrogate performance

Table 5 reports the relative L^2 error on the held-out test set (1620 simulations) for all 14 end-to-end surrogate checkpoints, evaluated on the physical field $U(t)$. All three model families (SVD, SLAE, LLAE) use the same surrogate MLP architecture (Appendix E), so the comparison isolates the effect of the spatial compression strategy rather than differences in the predictor capacity. Two additional columns facilitate interpretation: **Gap** is the difference between the surrogate median and the standalone AE reconstruction median from Table 3 — a positive value indicates overhead introduced by the surrogate step, while a negative value means that joint fine-tuning of the decoder actually **improves** upon the frozen-AE baseline; **vs SVD** is the difference relative to the matching SVD surrogate from Table 2 at the same (K, d_z, γ) , and is negative throughout, confirming that all AE surrogates outperform the linear baseline.

Overall accuracy. LLAE achieves 4.13–4.63% median L^2 error across all configurations, compared to 5.69–11.61% for SLAE. The best LLAE surrogate ($d_z = 128$, $K = 16$, $\gamma = 0.01$, 4.13%) represents a **54%** reduction in median error relative to the best POD surrogate (8.91%), while the best SLAE surrogate ($d_z = 64$, $K = 32$, $\gamma = 0.01$, 5.69%) achieves a **36%** reduction. Figure 9 summarises all 14 checkpoints sorted by median error.

End-to-end fine-tuning effect. The Gap column reveals a qualitative asymmetry between the two families. For LLAE, the surrogate step introduces a small positive overhead of +0.83 to +1.44 pp above the AE reconstruction floor: the surrogate approximation is not perfectly lossless, but the cost is modest given that the AE floor itself already saturates below 3.5%.

For SLAE, by contrast, the Gap is **negative** across all configurations, ranging from −0.40 to −3.14 pp — meaning that the end-to-end surrogate pipeline **outperforms** the standalone SLAE autoencoder. The mechanism is the following. During Phase 1 (AE training), the SLAE loss $\mathcal{L}_{\text{SLAE}}$ (equation (9), Section 2.2.1) is defined purely in the Laplace domain: the decoder is never directly supervised on physical-time reconstruction

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	Gap (pp)	vs POD (pp)
LLAE	128	16	0.01	4.13	4.48	6.82	+1.21	−7.16
	64	32	0.01	4.63	4.97	7.45	+1.44	−4.12
	64	16	0.01	4.18	4.52	6.84	+0.98	−4.73
	64	16	0.0	4.25	4.60	7.09	+0.99	−4.71
	64	16	0.001	4.21	4.53	6.91	+0.89	−4.73
	64	8	0.01	4.18	4.62	7.11	+0.83	−8.23
	32	16	0.01	4.31	4.75	7.27	+0.88	−7.38
SLAE	64	16	0.0	7.38	7.52	8.80	−0.40	−1.58
	64	16	0.001	7.42	7.56	8.89	−0.40	−1.52
	128	16	0.01	7.49	7.66	9.16	−0.49	−3.80
	64	32	0.01	5.69	6.06	7.97	−2.65	−3.06
	64	16	0.01	7.59	7.86	9.49	−1.33	−1.32
	32	16	0.01	7.62	7.84	9.49	−3.14	−4.07
	64	8	0.01	11.61	11.87	13.83	−0.65	−0.80

Table 5: Direct surrogate results on the test set (1620 simulations), same row order as Table 3. **Gap**: surrogate median – AE reconstruction median; negative values indicate that end-to-end fine-tuning improves upon the standalone AE. **vs POD**: surrogate median – matching POD surrogate median (Table 2); all values are negative. Best per family in bold.

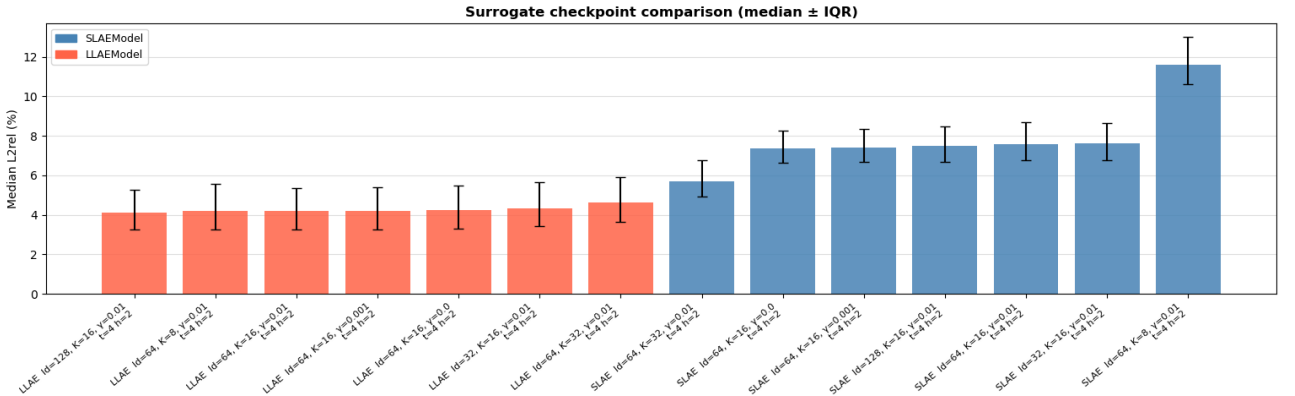


Figure 9: Surrogate checkpoint comparison: median L^2 relative error on the test set ($n = 1620$), sorted by median with IQR error bars. LLAE (red) achieves 4.1–4.6%, SLAE (blue) 5.7–11.6%. The best POD surrogate (8.91%) is indicated by a dashed horizontal line for reference.

quality, as computing a physical-time loss would require N_t -frame decoder passes and negate SLAE’s speed advantage. During Phase 2 (surrogate training), the combined loss $\mathcal{L}_{\text{total}}$ (Section 2.3) introduces a direct physical-time reconstruction term for the first time, and the decoder is jointly fine-tuned through it. This spatial supervision signal corrects residual errors that the Laplace-domain loss alone was unable to resolve. The effect is largest when the AE floor is highest: at $d_z = 32$ (-3.14 pp) and $K = 32$ (-2.65 pp), the spatial loss during surrogate training compensates for the harder compression task, lowering the error below the standalone AE baseline.

Hyperparameter sensitivity. LLAE surrogates remain highly robust: varying K from 8 to 32, d_z from 32 to 128, or γ from 0 to 0.01 all yield errors in a tight 4.13–4.63% band, consistent with the insensitivity already observed at the AE level (Section 3.3). SLAE is more sensitive: $K = 8$ collapses to 11.61% (the spectral truncation is too severe for the surrogate to compensate), while $K = 32$ is the best SLAE configuration precisely because the decoder, given access to 32 Laplace frequencies, can recover finer spatial structure during fine-tuning. Ablation violin plots are provided in Appendix C.

Inference latency. The surrogate MLP (architecture detailed in Appendix E) adds negligible latency to the AE decoding pipeline: measured on an NVIDIA RTX A5000, SLAE at $K = 16$ runs at 8.4 ± 0.1 ms per sample and LLAE at 18.3 ± 0.7 ms, virtually identical to the AE-only figures reported in Table 4 and consistent with the linear projection cost dominating over the network forward pass. SLAE therefore preserves its $\approx 2.2\times$ latency advantage over LLAE. Figure 10 details the latency breakdown across all 14 checkpoints.

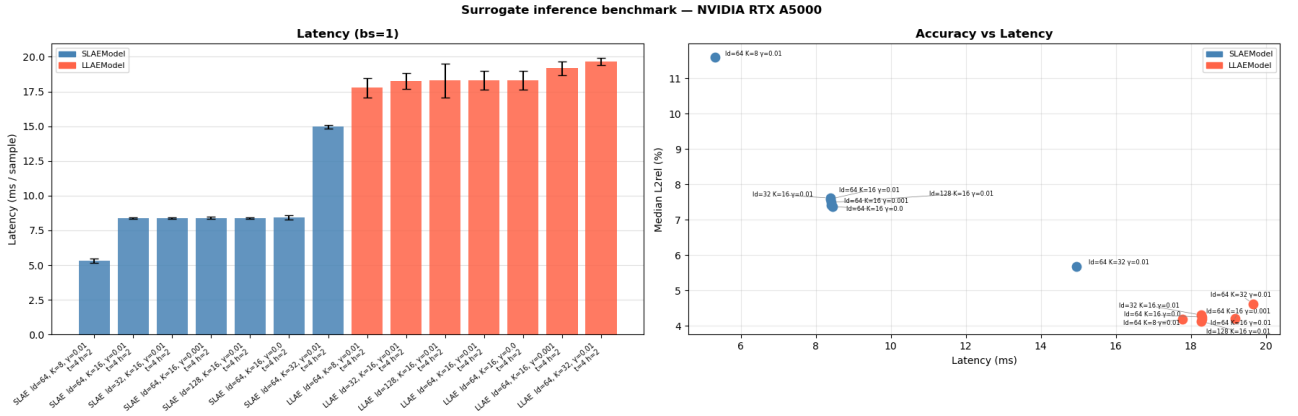


Figure 10: Surrogate inference benchmark (NVIDIA RTX A5000, 50 CUDA Event timed runs, batch size 1). Left: per-sample latency by model family and K . Right: accuracy-latency scatter (median L^2 error vs. latency).

Per-checkpoint error histograms for all 14 surrogate checkpoints are provided in Appendix A (Figure 22).

3.4.2 SVD surrogates: single-mode latent compression

The direct surrogate of Section 3.4.1 predicts the full d_z -dimensional latent at each frequency. To evaluate whether compressing that representation before learning the dynamics can instead improve the predictor’s accuracy, we assess a hybrid approach inspired by Rank Reduction Autoencoders (RRAE) [24]: projecting the latent trajectories

onto a truncated SVD basis before applying the surrogate model. We evaluate 26 AE+SVD surrogate checkpoints across the LLAE and SLAE families. In these models the encoder is frozen, whereas the decoder and the SVD basis V are trained end-to-end alongside the surrogate; the latent dimension is reduced to k_{svd} (either 16 or 32) via POD (SVD) before an MLP predicts the reduced coefficients from the physical parameters.

Table 6 reports the relative L^2 error on the held-out test set evaluated on the physical field $U(t)$.

Model	d_z	K	γ	k_{svd}	Median (%)	Mean (%)	p90 (%)
LLAE	64	16	0.01	32	4.24	4.61	6.75
	64	32	0.01	32	4.29	4.69	7.02
	64	16	0.001	32	4.44	4.96	7.41
	64	16	0.0	32	4.50	4.97	7.29
	64	8	0.01	32	4.60	5.04	7.47
	128	16	0.01	32	4.77	5.30	7.82
	32	16	0.01	16	5.55	6.29	9.39
	64	32	0.01	16	5.58	6.22	9.35
	128	16	0.01	16	5.83	6.51	9.48
	64	16	0.01	16	5.84	6.58	9.79
	64	16	0.001	16	5.90	6.72	9.92
	64	16	0.0	16	5.94	6.76	10.18
	64	8	0.01	16	6.54	7.37	11.05
	64	32	0.01	32	6.03	6.58	9.02
SLAE	64	32	0.01	16	6.96	7.80	11.24
	64	16	0.0	32	7.50	7.73	9.26
	64	16	0.001	32	7.62	7.88	9.52
	64	16	0.01	32	7.90	8.32	10.54
	32	16	0.01	16	7.95	8.38	10.71
	64	16	0.001	16	8.01	8.43	10.80
	128	16	0.01	32	8.12	8.57	10.98
	64	16	0.0	16	8.13	8.61	11.06
	64	16	0.01	16	8.71	9.40	12.68
	128	16	0.01	16	8.89	9.71	13.43
	64	8	0.01	32	11.64	11.93	13.96
	64	8	0.01	16	11.97	12.38	14.88

Table 6: AE+SVD surrogate results on the test set ($n = 1620$), evaluating the physical field reconstruction error using LLAE and SLAE autoencoders coupled with an intermediate SVD compression ($k_{\text{svd}} \in \{16, 32\}$). Best median per family in bold.

Does compressing the latent space help? The central question is whether predicting on a smaller, orthogonalised latent space simplifies the MLP’s task enough to offset the

information lost to SVD truncation. It does not. Measured **at matching autoencoder configuration**, the SVD bottleneck adds a small penalty over the direct surrogate for both families: fixing $d_z = 64$, $\gamma = 0.01$, the best LLAE-SVD ($K = 16$, $k_{\text{svd}} = 32$, 4.24%) sits +0.06 pp above its direct counterpart (4.18%), and the best SLAE-SVD ($K = 32$, $k_{\text{svd}} = 32$, 6.03%) +0.34 pp above its own (5.69%). The MLP navigates the full d_z -dimensional latent space perfectly well, so the SVD projection merely discards spatial detail the direct surrogate had already learned to predict — compressing the latent before the surrogate degrades the reconstruction rather than helping it. Figure 11 collects the per-checkpoint medians.

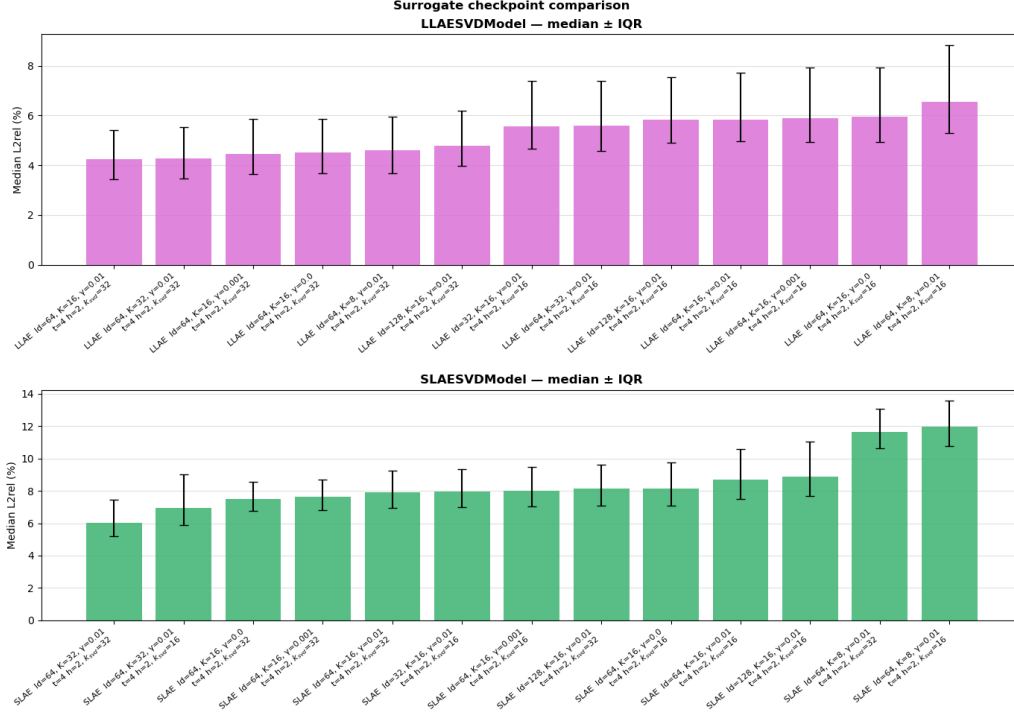


Figure 11: AE+SVD surrogate checkpoint comparison: median L^2 relative error with IQR error bars, grouping by model family. LLAE achieves 4.2–6.5%, SLAE 6.0–12.0%.

Impact of k_{svd} . To quantify the benefit of retaining a larger number of SVD modes, we compare pairs of configurations differing only in k_{svd} (16 vs. 32). Figure 12 highlights that setting $k_{\text{svd}} = 32$ systematically improves the surrogate’s accuracy across all models (all points lie below the $y = x$ diagonal). The gain is significantly more pronounced for LLAE models (between +1.06 and +1.94 pp reduction in median error) than for SLAE models (+0.33 to +0.93 pp), indicating that LLAE latent spaces contain more useful fine-grained information captured by the additional 16 SVD modes, whereas SLAE may inherently discard spatial details at the AE stage, limiting the potential gains of a larger k_{svd} .

Hyperparameter sensitivity. Sensitivity to the AE hyperparameters K , d_z , and γ mirrors the direct case (Section 3.4.1): SLAE again degrades sharply at $K = 8$ (11.6–12.0%), while LLAE stays within a tight band. The lever specific to this pipeline is k_{svd} , whose effect is quantified above: retaining 32 rather than 16 modes is uniformly beneficial, and more so for LLAE than for SLAE.

Inference latency. Introducing the SVD decoding step (matrix multiplication) between the MLP prediction and the spatial decoder adds a slight computational overhead.

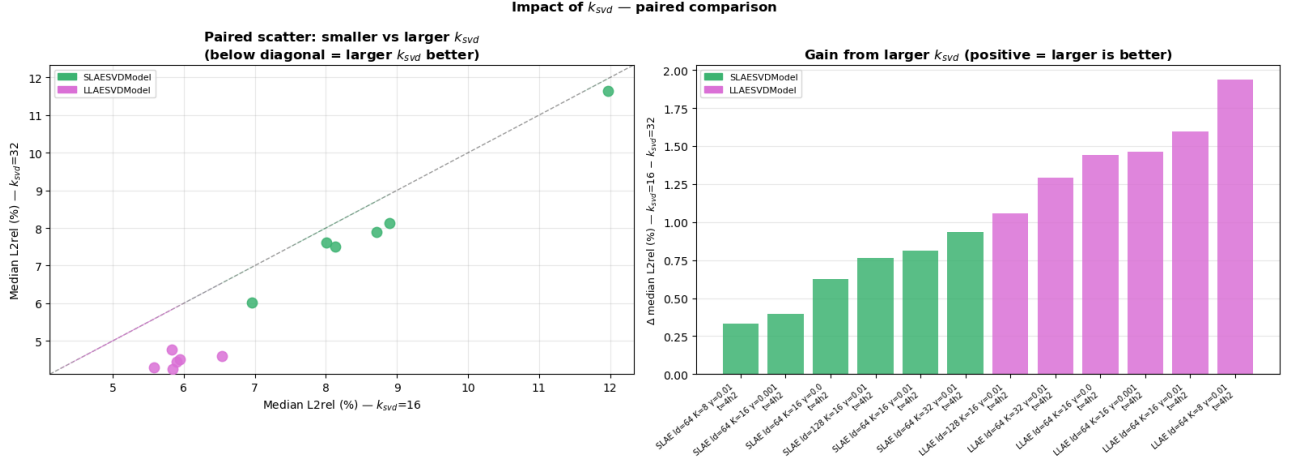


Figure 12: Impact of k_{svd} on SVD surrogate performance. Left: Scatter of paired median errors showing $k_{\text{svd}} = 32$ consistently outperforms $k_{\text{svd}} = 16$. Right: Absolute error reduction when shifting from 16 to 32 modes, sorted by magnitude.

Measured on an NVIDIA RTX A5000 (batch size 1), LLAE-SVD requires approximately 22 ms per sample, while SLAE-SVD requires about 9 ms per sample. The SLAE-SVD model maintains its pronounced speed advantage ($\approx 2.4\times$) over LLAE-SVD, reflecting its lighter decoder architecture. The trade-off between latency and median error for AE+SVD surrogates is presented in Figure 13.

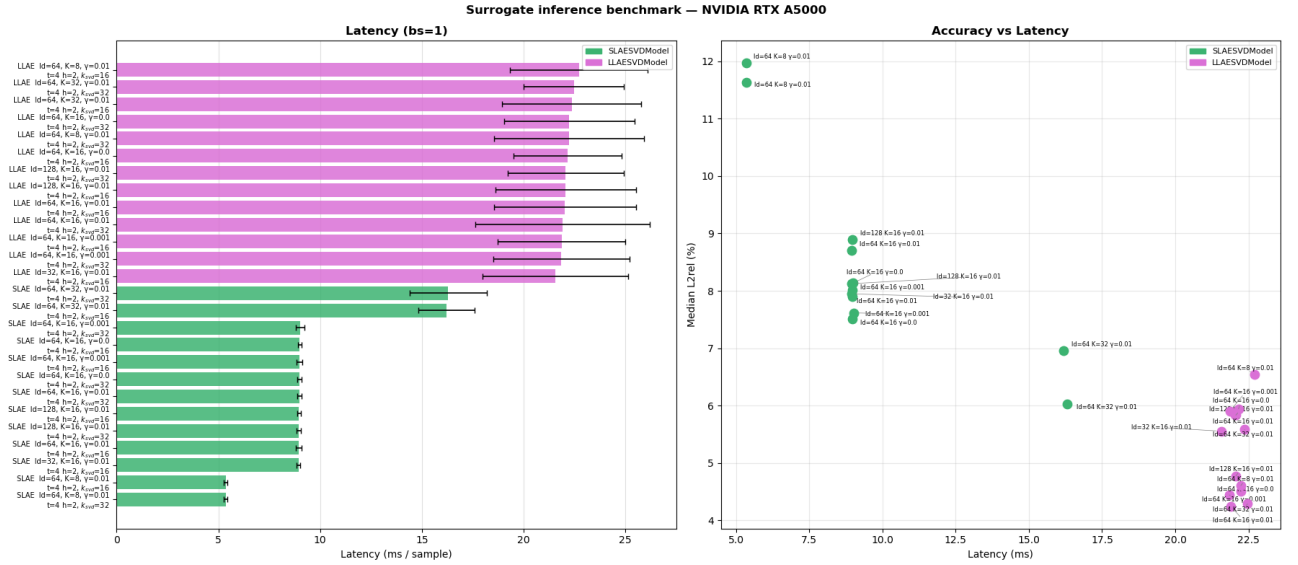


Figure 13: AE+SVD surrogate inference benchmark (NVIDIA RTX A5000). Left: per-sample latency across checkpoints. Right: accuracy–latency scatter representing median L^2 error vs. latency.

3.4.3 Tucker surrogates: joint-mode latent compression

Section 3.4.2 compressed the latent representation along a single mode. We now evaluate the Tucker variant of Section 2.3, which factorises the frequency and latent modes **jointly**: the surrogate predicts a compact core matrix $\hat{\mathcal{G}} \in \mathbb{C}^{r_s \times r_z}$ (real-valued for SLAE) instead of K independent latents of dimension d_z , back-projecting through the frozen HOOI factors $U^{(s)}, U^{(z)}$ before decoding. We sweep the Tucker ranks over a grid $r_s \in \{4, 8\}$

(frequency mode, $K = 16 \rightarrow r_s) \times r_z \in \{8, 16, 32\}$ (latent mode, $d_z = 64 \rightarrow r_z$), for both families, yielding 12 checkpoints. All share the same Phase 1 autoencoder ($d_z = 64$, $K = 16$, $\gamma = 0.01$); only the offline factors and the surrogate differ. The compression ratio $\rho = r_s r_z / (K d_z)$ measures the reduction of the surrogate target dimension relative to the direct pipeline (which predicts $K d_z = 1024$ scalars per sample).

Beyond the physical-field error $\varepsilon_{\text{field}}$ (relative L^2 on $U(t)$, full pipeline), we report two decoder-free diagnostics computed in the latent space, which split the difficulty between the offline compression and the surrogate MLP:

$$(38) \quad \varepsilon_{\text{trunc}} = \frac{\|\hat{Z} - \Pi \hat{Z}\|}{\|\hat{Z}\|}, \quad \varepsilon_{\text{reg}} = \frac{\|\hat{\mathcal{G}}_{\text{pred}} - \hat{\mathcal{G}}_{\text{true}}\|}{\|\hat{\mathcal{G}}_{\text{true}}\|},$$

where $\Pi = U^{(s)} U^{(s)\top} (\cdot) U^{(z)} U^{(z)\top}$ is the Tucker projector. $\varepsilon_{\text{trunc}}$ measures the energy discarded by the rank truncation (factors only, independent of the surrogate), while ε_{reg} measures how well the MLP predicts the true core (surrogate only, independent of the truncation). Table 7 reports all three on the held-out test set.

Model	r_s	r_z	ρ	Median (%)	Mean (%)	p90 (%)	$\varepsilon_{\text{trunc}}$ (%)	ε_{reg} (%)	Lat (ms)
LLAE	8	32	0.250	4.33	4.75	7.16	36.98	7.25	18.2
	4	32	0.125	4.57	5.04	7.61	40.34	5.67	18.0
	8	16	0.125	4.87	5.43	8.11	61.86	4.38	18.3
	4	16	0.062	5.16	5.77	8.70	61.59	3.83	18.0
	8	8	0.062	6.85	7.78	11.75	81.21	4.99	18.3
	4	8	0.031	6.87	8.02	12.73	81.03	3.80	18.0
SLAE	8	32	0.250	7.92	8.35	10.55	42.40	6.76	7.4
	4	32	0.125	8.15	8.68	11.14	50.82	6.58	7.1
	4	16	0.062	8.68	9.36	12.79	69.59	5.18	7.1
	8	16	0.125	8.85	9.51	12.89	67.59	5.83	7.5
	4	8	0.031	9.93	10.92	15.61	80.63	5.70	7.1
	8	8	0.062	9.96	11.05	16.00	80.05	7.16	7.5

Table 7: Tucker surrogate results on the test set ($n = 1620$), sorted by median error within each family. $\rho = r_s r_z / (K d_z)$ is the compression ratio of the surrogate target relative to the direct pipeline. $\varepsilon_{\text{trunc}}$: median latent-energy discarded by the rank truncation (factors only). ε_{reg} : median relative error of the predicted core (surrogate only). **Lat**: GPU inference latency at batch size 1 (NVIDIA RTX A5000). Best median per family in bold.

Overall accuracy. As with SVD, the relevant question is what the extra compression costs relative to the direct and single-mode pipelines **at matching autoencoder configuration**. Fixing the shared AE ($d_z = 64$, $K = 16$, $\gamma = 0.01$), the median error grows only gently along the compression axis: $4.18\% \rightarrow 4.24\% \rightarrow 4.33\%$ for LLAE (direct $\rightarrow$ SVD with $k_{\text{svd}} = 32 \rightarrow$ Tucker with $r_s = 8$, $r_z = 32$), and $7.59\% \rightarrow 7.90\% \rightarrow 7.92\%$ for SLAE. Tucker thus adds at most $+0.15$ pp over the direct surrogate and $+0.09$ pp over SVD — while shrinking the surrogate target to $r_s r_z = 256$ scalars, a $4\times$ reduction over the direct

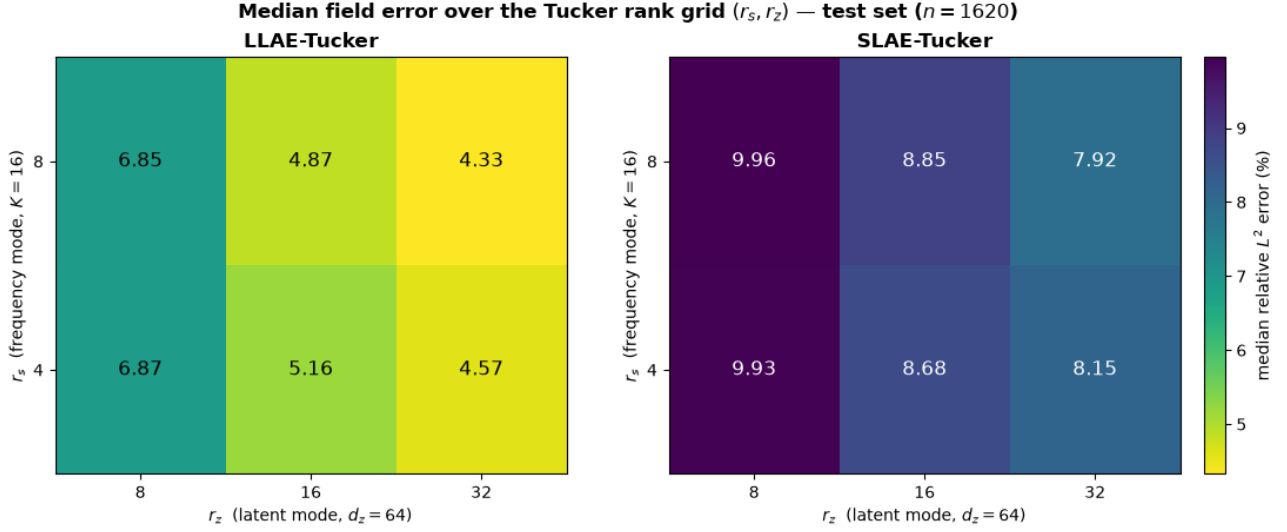


Figure 14: Median field error over the Tucker rank grid (r_s, r_z) (test set, $n = 1620$), for LLAE-Tucker (left) and SLAE-Tucker (right). Both panels share the same colour scale. The latent-mode rank r_z (horizontal) drives the error far more than the frequency-mode rank r_s (vertical).

target ($Kd_z = 1024$) and a $2\times$ reduction over the SVD target ($Kk_{\text{svd}} = 512$). Factorising both modes at once is what buys this: exploiting cross-frequency correlations lets Tucker match the single-mode SVD accuracy at half the target dimension.

Rank sensitivity: r_z dominates r_s . The grid of Figure 14 is strongly anisotropic, and this is the most informative outcome of the sweep. Increasing the latent-mode rank r_z from 8 to 32 nearly halves the LLAE error ($6.85\% \rightarrow 4.33\%$ at $r_s = 8$), and the SLAE error falls in step ($9.96\% \rightarrow 7.92\%$). Doubling the frequency-mode rank r_s from 4 to 8, by contrast, barely moves either ($4.57\% \rightarrow 4.33\%$ for LLAE, $8.15\% \rightarrow 7.92\%$ for SLAE at $r_z = 32$) — an effect an order of magnitude smaller. The rank sweeps of Figure 15 confirm the pattern distributionally: the r_z violins separate cleanly while the r_s violins overlap almost completely.

The interpretation reaches beyond this ablation. A rank-4 basis over the frequency mode already captures almost all of the joint variation of the $K = 16$ Laplace coefficients across the training set: the spectral representation is redundant enough that discarding three quarters of its dimensions costs nothing measurable. The $d_z = 64$ latent channels behave in the opposite way — they carry richer, less redundant information that a rank-8 basis truncates too aggressively. A tensor factorisation, fitted blind to the physics on training statistics alone, thus rediscovers the premise of this work: the Laplace representation is intrinsically compact along the frequency axis, and $K = 16$ already sits close to its useful floor. That this evidence arrives from a direction independent of the argument which motivated the transform is what makes it worth reporting.

In practice $r_s = 4$ is the parameter-efficient choice: it halves the surrogate size (1.35 M vs. 1.9 M parameters for $r_s = 8$) at a negligible accuracy cost. One caveat bounds the claim: the frequency axis is sampled at two points only ($r_s \in \{4, 8\}$), so the sweep establishes that the error is insensitive to r_s over that range, not that it has saturated. The uncompressed limit $r_s = K$ is structurally the single-mode compression of Section 3.4.2, although the two differ in the nature of the basis — real and trained there, complex and frozen here.

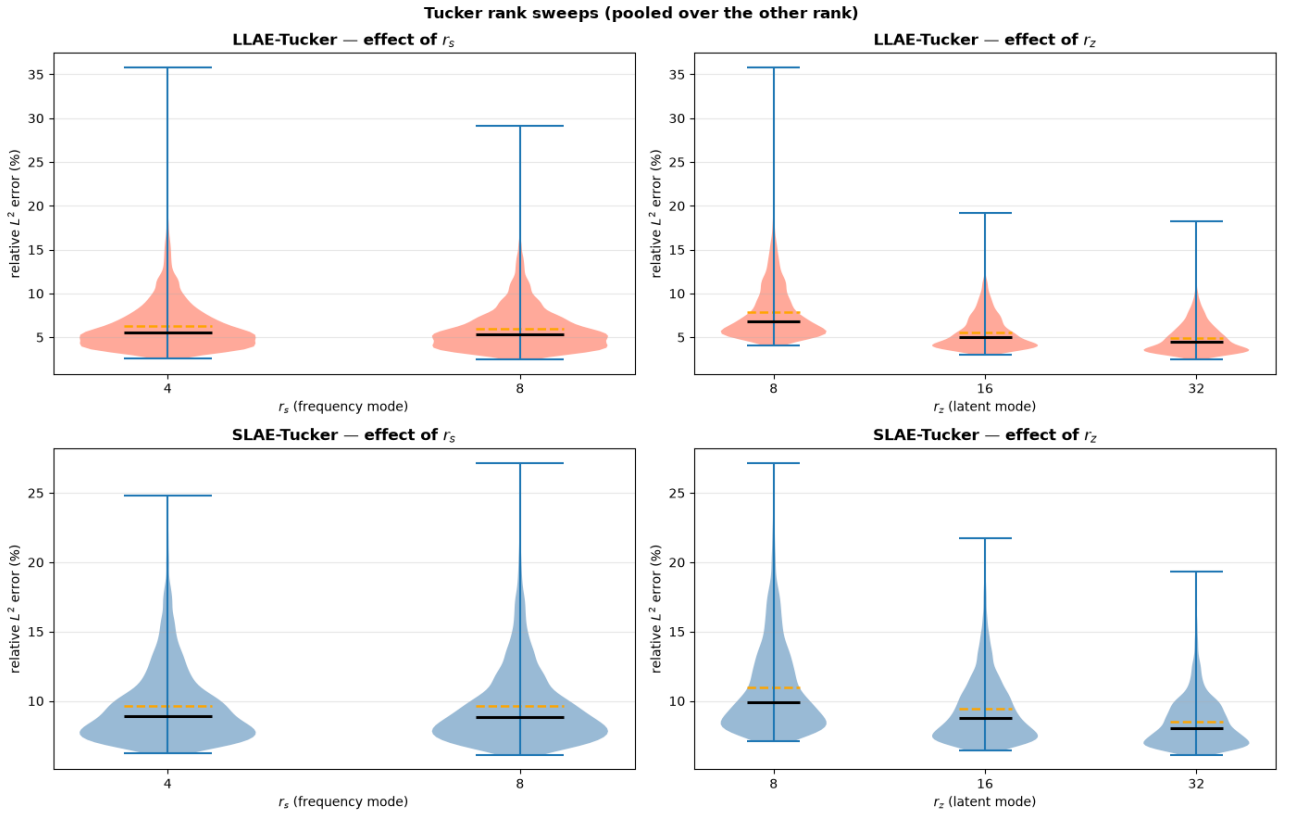


Figure 15: Tucker rank sweeps (test set, violin plots pooled over the other rank; black line = median, dashed orange = mean). Top row LLAE-Tucker, bottom row SLAE-Tucker; left column effect of r_s , right column effect of r_z . The latent-mode rank r_z separates the distributions, the frequency-mode rank r_s does not.

Truncation vs regression. The two decoder-free diagnostics of Table 7, shown in Figure 16, locate the accuracy bottleneck. The regression error ε_{reg} stays small throughout (3.8–7.3%): the MLP predicts the compact core reliably, and it grows only mildly as the core enlarges ($r_z = 32$ is harder to fit than $r_z = 8$). The truncation error $\varepsilon_{\text{trunc}}$, by contrast, is large and is what the field error tracks: it falls from $\sim 81\%$ at $r_z = 8$ to $\sim 37\%$ at $r_z = 32$ (LLAE), mirroring the field-error drop. Note that $\varepsilon_{\text{trunc}}$ is a **latent-space energy**, not a field error — discarding 80% of the latent norm costs only $\sim 7\%$ on $U(t)$, because the removed Tucker directions carry little decodable information. Its **variation** with the ranks, not its absolute value, is what governs accuracy. The conclusion is that the Tucker compression, not the surrogate, is the binding constraint: the MLP is never the limiting factor, so any accuracy gain must come from a richer latent basis (larger r_z) rather than a larger predictor.

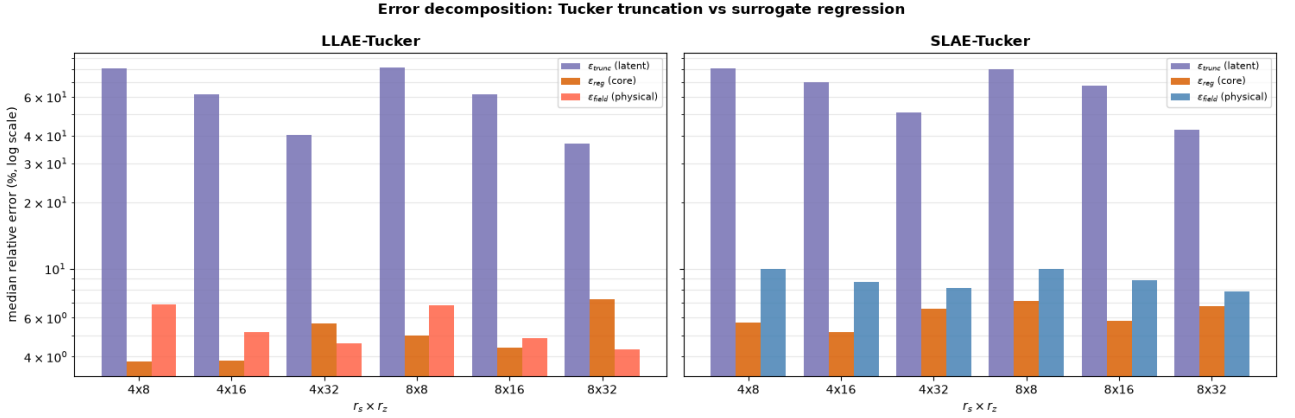


Figure 16: Error decomposition for LLAE-Tucker (left) and SLAE-Tucker (right): median latent truncation error $\varepsilon_{\text{trunc}}$, core regression error ε_{reg} , and physical-field error $\varepsilon_{\text{field}}$ for each (r_s, r_z) pair (log scale). The field error follows the truncation error, not the regression error.

Compression trade-off. Figure 17 plots the field error against the compression ratio ρ and against the surrogate size. Along the Pareto front, LLAE-Tucker degrades gracefully as the target is compressed: $r_s = 4$, $r_z = 32$ retains 4.57% median error at $\rho = 0.125$ (an $8\times$ smaller target than the direct surrogate), and $r_s = 4$, $r_z = 16$ still reaches 5.16% at $\rho = 0.062$ ($16\times$). Tucker therefore offers a tunable accuracy/compactness dial that the direct and SVD surrogates lack: when the surrogate output must be kept small — for storage, for a lightweight predictor, or for downstream many-query use — Tucker sustains near-direct accuracy at a fraction of the target dimension. It does not, however, **beat** the direct surrogate on raw accuracy; as with SVD (Section 3.4.2), reducing the latent space before the surrogate discards information the direct predictor is capable of learning.

Decoder co-adaptation. A subtle consequence of the end-to-end fine-tuning (Section 3.4.1) is worth flagging. Because the decoder is jointly updated during surrogate training ($\text{lr_decoder} = 5 \times 10^{-5}$), it can co-adapt to the distribution of **predicted** cores rather than the true ones. We probe this by decoding the exact projected core in place of the surrogate’s prediction: if the decoder had not co-adapted, the exact core would yield a lower error and serve as a valid oracle lower bound. For LLAE-Tucker the substitution is neutral (+0.07 pp), so the oracle bound holds. For SLAE-Tucker, however, decoding the exact core **raises** the error sharply (+11.1 pp on the probe subsample at $r_s = 8$,

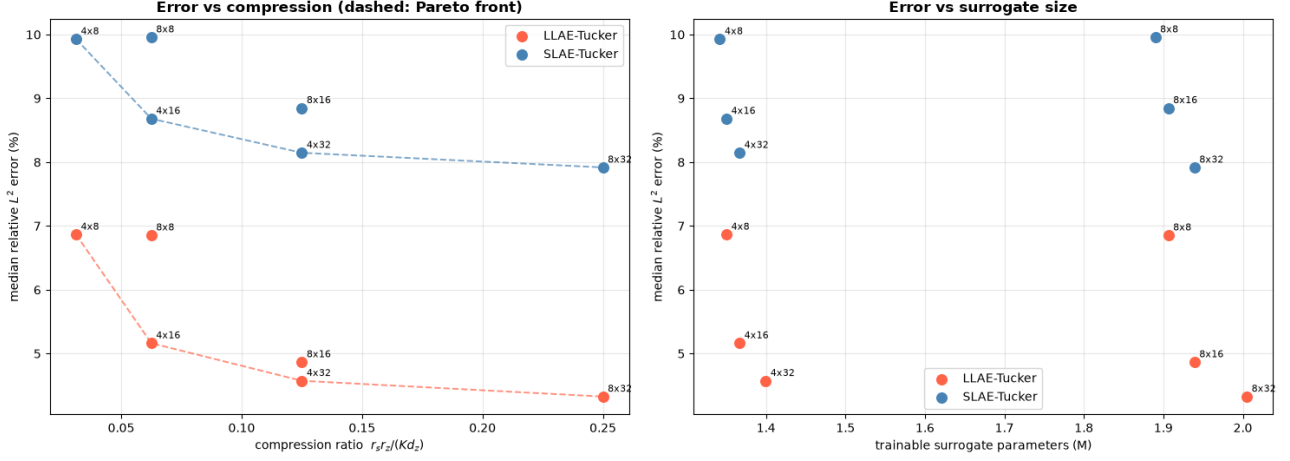


Figure 17: Left: median field error vs. compression ratio $\rho = r_s r_z / (K d_z)$ (dashed line = Pareto front per family; points sharing a ratio differ in (r_s, r_z)). Right: median field error vs. trainable surrogate parameters.

$r_z = 16$): the SLAE decoder has specialised to the predicted-core distribution, exactly the mechanism that produced the negative surrogate Gap for SLAE in Section 3.4.1. A physical-space oracle is therefore not a valid lower bound for SLAE-Tucker, which is why the truncation and regression diagnostics above are computed in the latent space, where they remain decoder-independent.

Inference latency. The Tucker back-projection $(U^{(s)})^\top \hat{G} U^{(z)\top}$, two small matrix products) is negligible in the inference budget. Measured on an NVIDIA RTX A5000 at batch size 1 (Figure 18), LLAE-Tucker runs at ~ 18 ms per sample and SLAE-Tucker at ~ 7 ms, essentially independent of the ranks and virtually identical to the direct and SVD pipelines. SLAE-Tucker thus preserves the $\approx 2.4\times$ latency advantage of its lighter decoder, at the cost of the higher error floor discussed above.

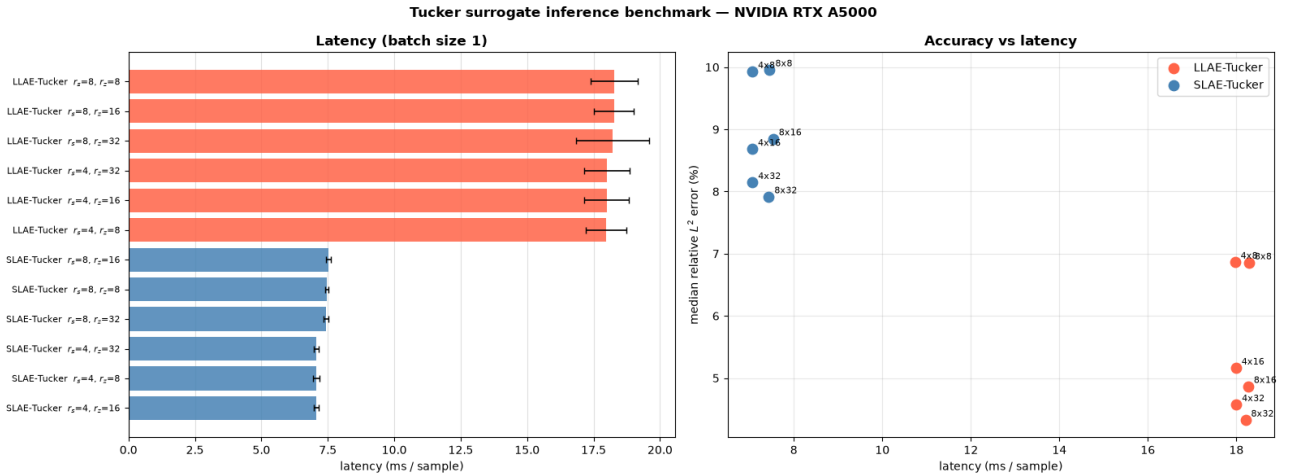


Figure 18: Tucker surrogate inference benchmark (NVIDIA RTX A5000, 50 CUDA Event timed runs, batch size 1). Left: per-checkpoint latency. Right: accuracy–latency scatter. Latency is set by the decoder family, not the Tucker ranks.

Per-checkpoint error histograms for all 12 Tucker surrogates are provided in Appendix A (Figure 23).

3.5 Optimal laplace contour choice

4 Comparison and Discussion

4.1 Global synthesis

4.2 SLAE vs LLAE: a trade-off between speed and accuracy

4.3 Is latent compression worth it?

Sections 3.4.1–3.4.3 evaluated three surrogate strategies — direct, single-mode SVD, and joint-mode Tucker — that differ only in how the latent prediction target is compressed. Do the compressed variants gain anything? On inference cost, the answer is no, and the reason is structural: the three pipelines share the same decoder, and it is the decoder — reconstructing the K Laplace frames (SLAE) or all N_t time frames (LLAE) — that dominates inference. SVD and Tucker change only the width of the MLP output head and insert a small linear back-projection ($V^\top$, or the two Tucker factors) before decoding; neither touches the decoder workload. Latency is therefore effectively invariant to the compression strategy: within each family the three pipelines fall in the same band (SLAE ~ 7 – 9 ms, LLAE ~ 18 – 22 ms at batch size 1 on the RTX A5000), the SVD variant being marginally the slowest because of its per-frequency back-projection. Table 8 places accuracy, target dimension and latency side by side at the shared AE configuration.

Family	Strategy	Target (coeff.)	Median (%)	Latency (ms)
LLAE	Direct	1024	4.18	18.3
	SVD ($k_{\text{svd}} = 32$)	512	4.24	~ 22
	Tucker ($r_s=8, r_z=32$)	256	4.33	18.2
SLAE	Direct	1024	7.59	8.4
	SVD ($k_{\text{svd}} = 32$)	512	7.90	~ 9
	Tucker ($r_s=8, r_z=32$)	256	7.92	7.4

Table 8: Direct vs. SVD vs. Tucker at the shared autoencoder configuration ($d_z = 64$, $K = 16$, $\gamma = 0.01$). **Target (coeff.)**: number of latent coefficients the surrogate predicts per sample (Kd_z , Kk_{svd} , $r_s r_z$ respectively; complex-valued for LLAE, real for SLAE). Median relative L^2 error on the test set ($n = 1620$); latency at batch size 1 on the NVIDIA RTX A5000. Compression shrinks the target by up to $4\times$ but leaves accuracy and latency essentially unchanged.

The verdict is clear. Relative to the direct surrogate, neither SVD nor Tucker improves accuracy (each adds a fraction of a point) nor latency (the pipeline is decoder-bound). What they **do** buy is a smaller, simpler prediction target — Tucker down to 256 coefficients, a $4\times$ reduction over the direct $Kd_z = 1024$. That compactness only pays off when the surrogate, not the decoder, is the binding constraint: a tight training-data budget, storage or transmission of the predicted coefficients, or downstream analysis of the compact Tucker core. For the present problem, where the decoder governs both cost and error, the **direct** surrogate is the pragmatic choice; the value of SVD and Tucker is chiefly methodological, showing that most of the latent’s predictive content survives aggressive rank reduction.

4.4 The importance of choosing an optimal Laplace contour

4.5 Positioning with respect to related work

Our pipeline reuses well-established components and combines them along a new axis. The **spatial** compressor is a convolutional encoder–decoder applied directly to the field, i.e. the deep convolutional-autoencoder ROM of Lee and Carlberg [18] and of the original DL-ROM [5]; we deliberately omit the preliminary POD reduction of POD-DL-ROM [6], which is an efficiency device (it lowers the autoencoder’s training cost) rather than an accuracy device, and is unnecessary for a convolutional network operating on a structured grid. The **linear** baseline against which we measure the nonlinear gain is a POD basis followed by a parameter-to-coefficient network, that is, the non-intrusive POD-NN of Hesthaven and Ubbiali [14], here applied frequency-by-frequency in the Laplace domain. Our optional SVD and Tucker stages (Sections 2.3.2–2.3.3) also use linear/multilinear projections, but in a role opposite to POD-DL-ROM: they act **after** the nonlinear autoencoder, in the latent space, to shrink the surrogate target — a post-compression, not a spatial pre-reduction.

What distinguishes our approach is not the spatial compressor but the treatment of **time**. All the DL-ROMs above keep the full temporal resolution of the trajectory: the reduced dynamics are queried once per time step through a (μ, t) -conditioned network [5, 6] or propagated sequentially in time [25], so the temporal work scales with N_t . We instead compress the time axis itself with the Laplace transform, predicting $K \ll N_t$ spectral coefficients in a single shot and recovering the full trajectory by a cheap linear inversion. This exploits the same spectral intuition as Laplace-domain linear ROMs [28, 9] and the Laplace Neural Operator [2], but transposes it into the **nonlinear, parameter-to-field** DL-ROM setting, where — to our knowledge — temporal Laplace compression has not previously been used. The Laplace stage is therefore orthogonal to the spatial-compression choices above and could in principle be layered on top of any of them, including POD-DL-ROM.

Relation to neural operators. The learnable-contour variant of LLAE (Section 2.4) deserves a closer comparison with operator-learning methods, which it resembles along two axes. First, it applies its spectral transform **inside a learned low-dimensional latent space** rather than on the physical field, exactly the design principle of the Latent Neural Operator [29]: encode to a compact latent, act there, decode back. Second, and more specifically, learning the complex poles $s_k = \sigma_k + i\omega_k$ end-to-end is a latent-space counterpart of the learnable pole–residue representation at the heart of the Laplace Neural Operator [2]; in both cases the temporal (or trajectory) dynamics are captured by a small set of trainable poles in the complex plane rather than by a fixed spectral grid or by explicit time-stepping. The surrogate itself reinforces the analogy: its shared trunk conditioned on a sinusoidal encoding of the frequency s_k (Figure 2) maps $(\mu, s_k) \mapsto \hat{z}(s_k)$, so the Laplace frequency plays the role of the output coordinate much as the trunk coordinate does in a DeepONet [20]. The essential difference is one of **setting**, not of mechanism: genuine neural operators are field-to-field maps between function spaces, discretization-invariant in their input, whereas our input is a finite parameter vector μ and our spatial decoder is a fixed-resolution convolutional network. LLAE with a learnable contour is thus best understood as a **parameter-conditioned, latent Laplace operator** — importing the learnable-pole and latent-operator ideas from [2, 29] into the DL-ROM regime, rather than as a neural operator in the strict field-to-field sense.

4.6 Discussion and perspectives

Appendices

A Per-Checkpoint Error Distributions

POD reconstruction. Figure 19 shows the distribution of relative L^2 errors for each of the 7 POD reconstruction configurations evaluated in Section 3.2.

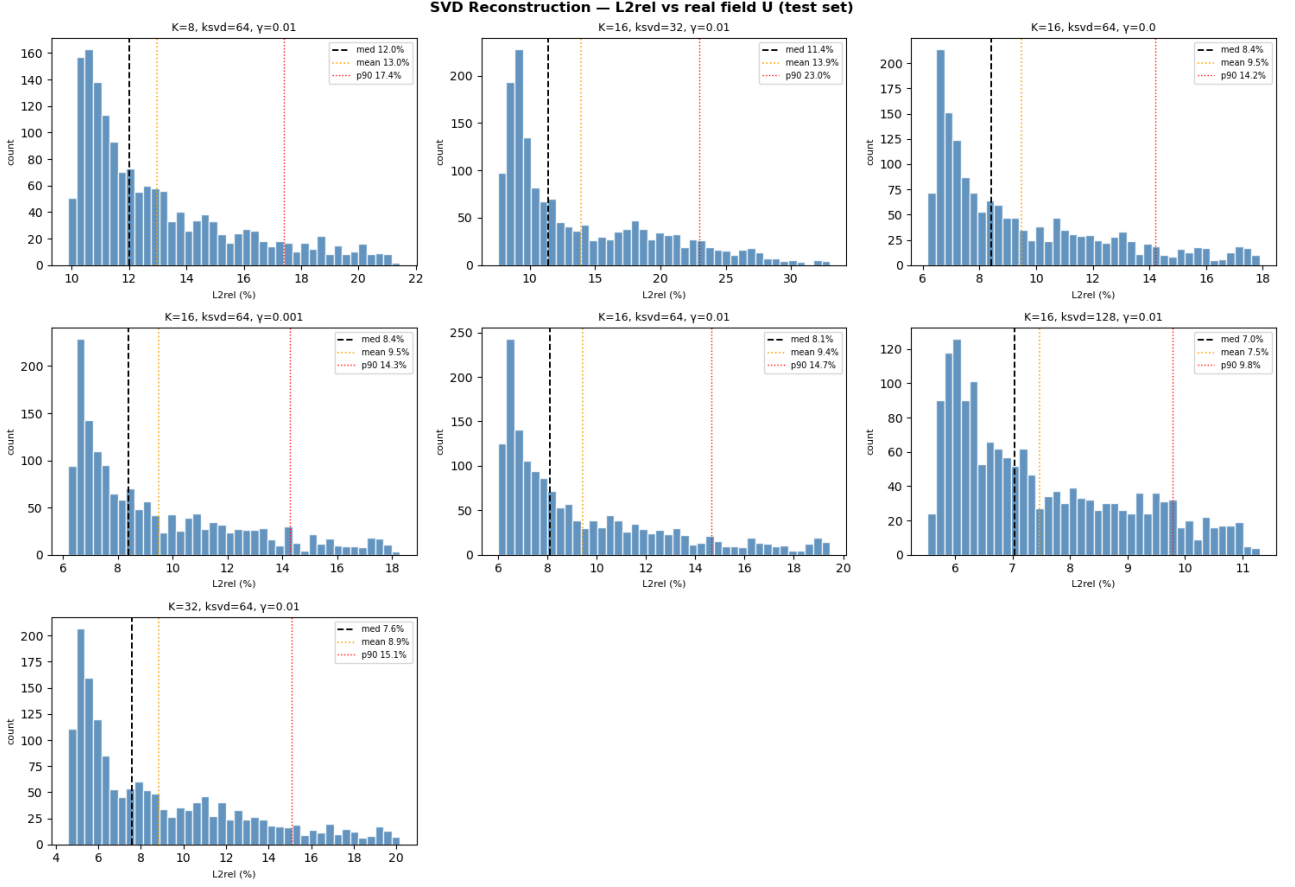


Figure 19: Per-configuration error histograms for the POD reconstruction (test set, $n = 1620$). The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

POD surrogate. Figure 20 shows the corresponding distributions for the end-to-end POD surrogate (Section 3.2).

AE checkpoints. Figure 21 shows the distribution of relative L^2 errors across the 1620 test simulations for each of the 14 AE checkpoints evaluated in Section 3.3. Dashed lines indicate the median (black) and mean (orange); the dotted red line marks the 90th percentile.

Surrogate checkpoints. Figure 22 shows the per-sample error distribution for each of the 14 direct surrogate checkpoints evaluated in Section 3.4.1.

Tucker surrogate checkpoints. Figure 23 shows the per-sample error distribution for each of the 12 Tucker surrogate checkpoints evaluated in Section 3.4.3.

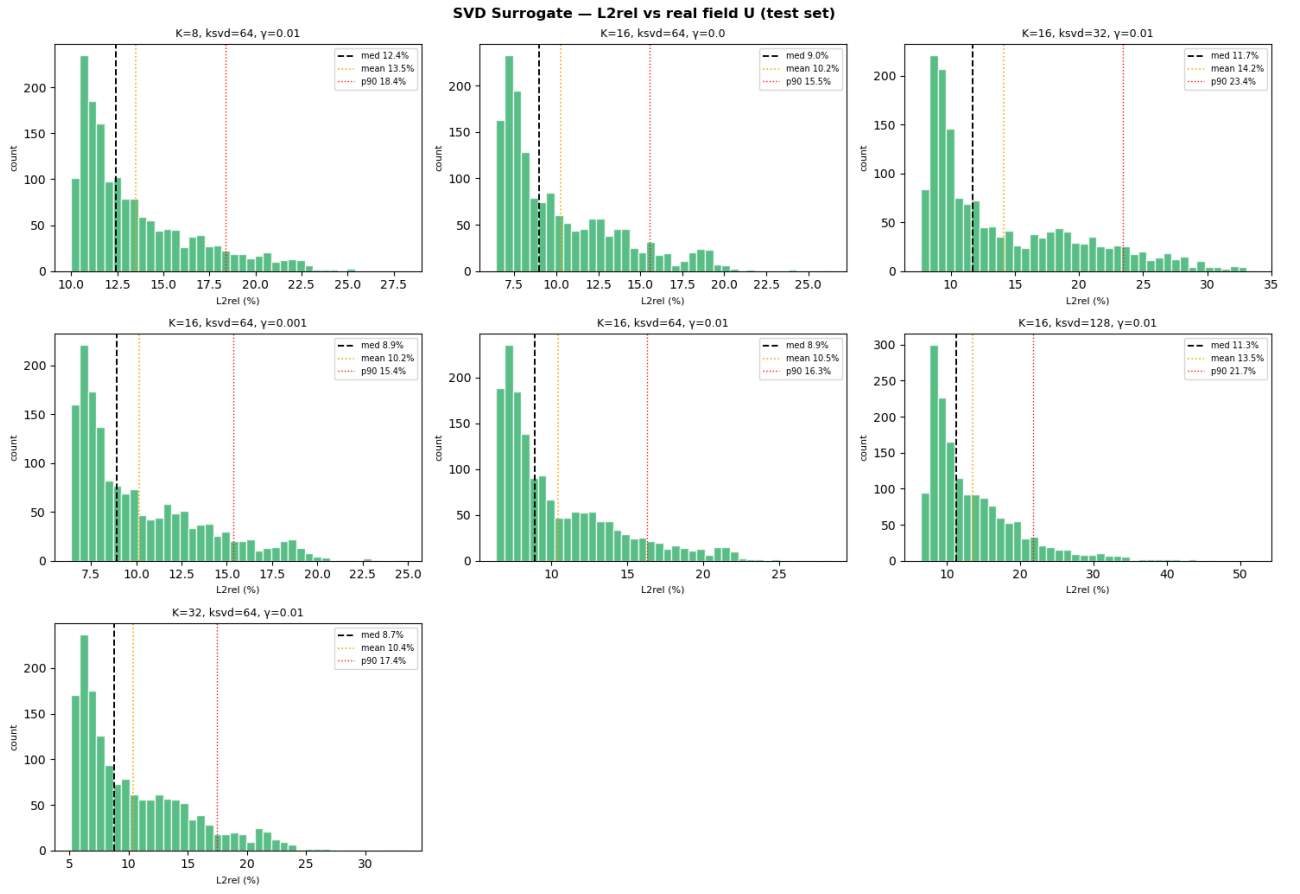


Figure 20: Per-configuration error histograms for the POD surrogate (test set, $n = 1620$). Same layout as Figure 19; green bars indicate surrogate predictions.

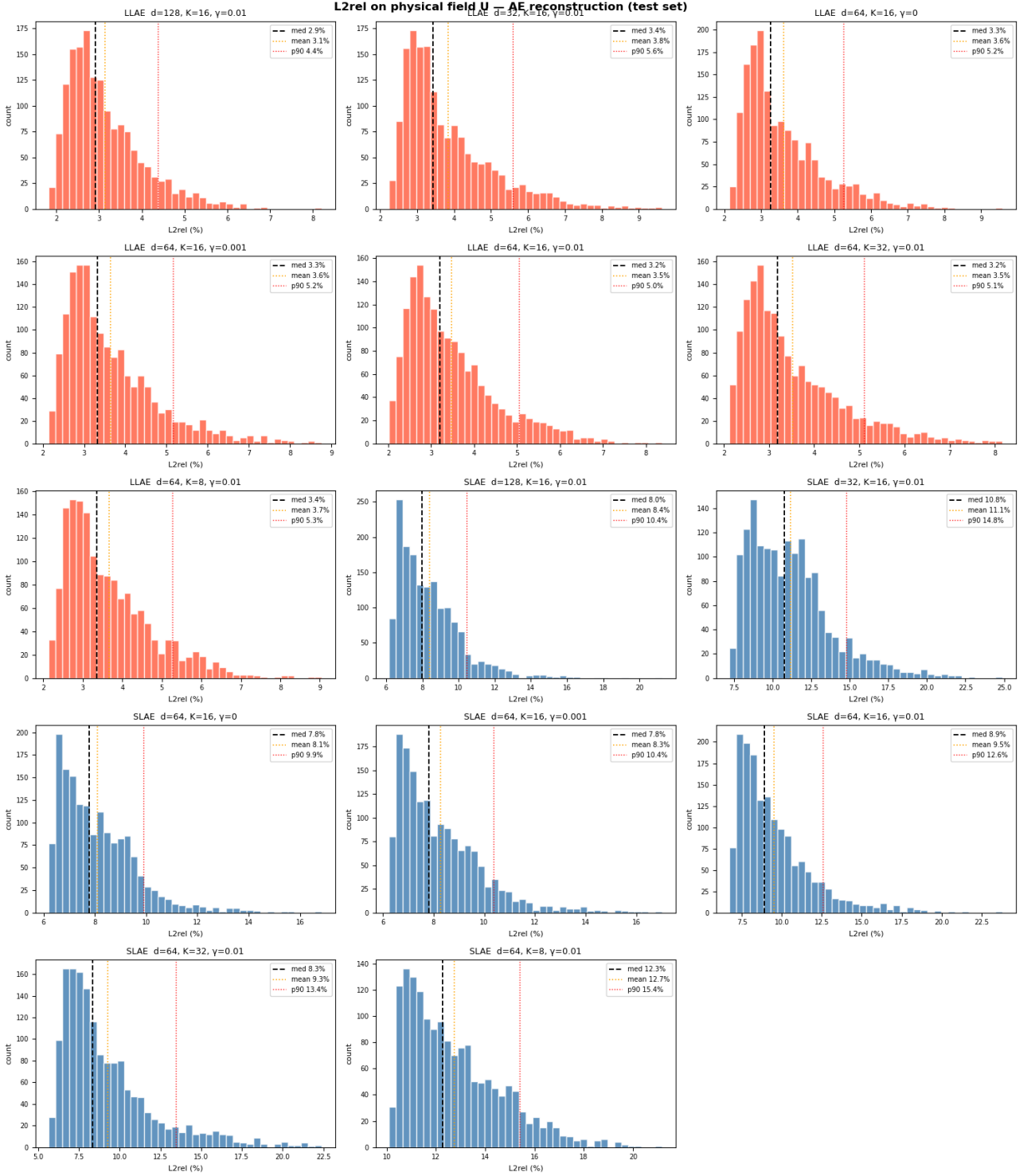


Figure 21: Per-checkpoint error histograms on the test set ($n = 1620$). Each panel corresponds to one checkpoint; SLAE in blue, LLAE in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

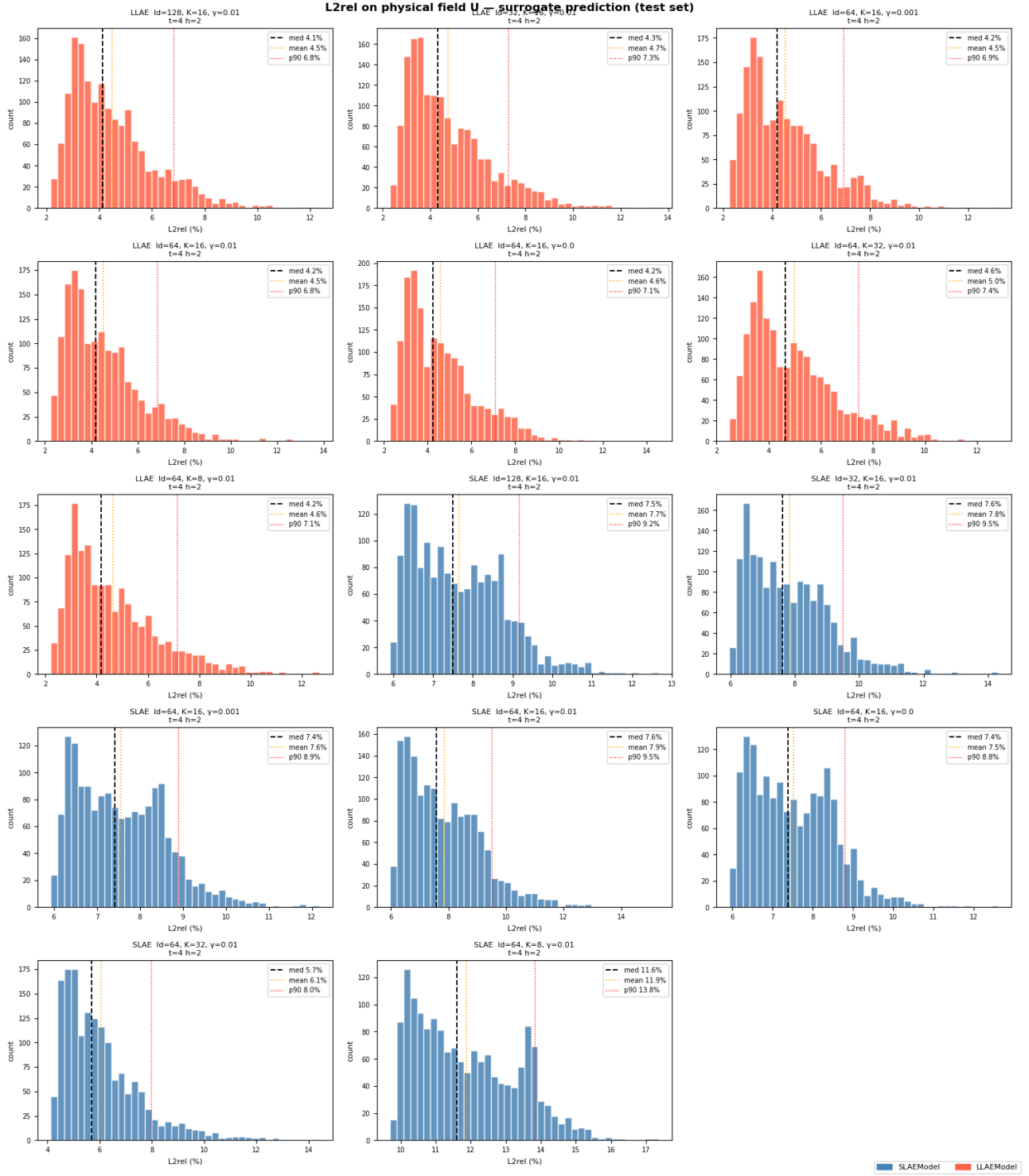


Figure 22: Per-checkpoint error histograms for all 14 direct surrogate models (test set, $n = 1620$). SLAE checkpoints in blue, LLAE checkpoints in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

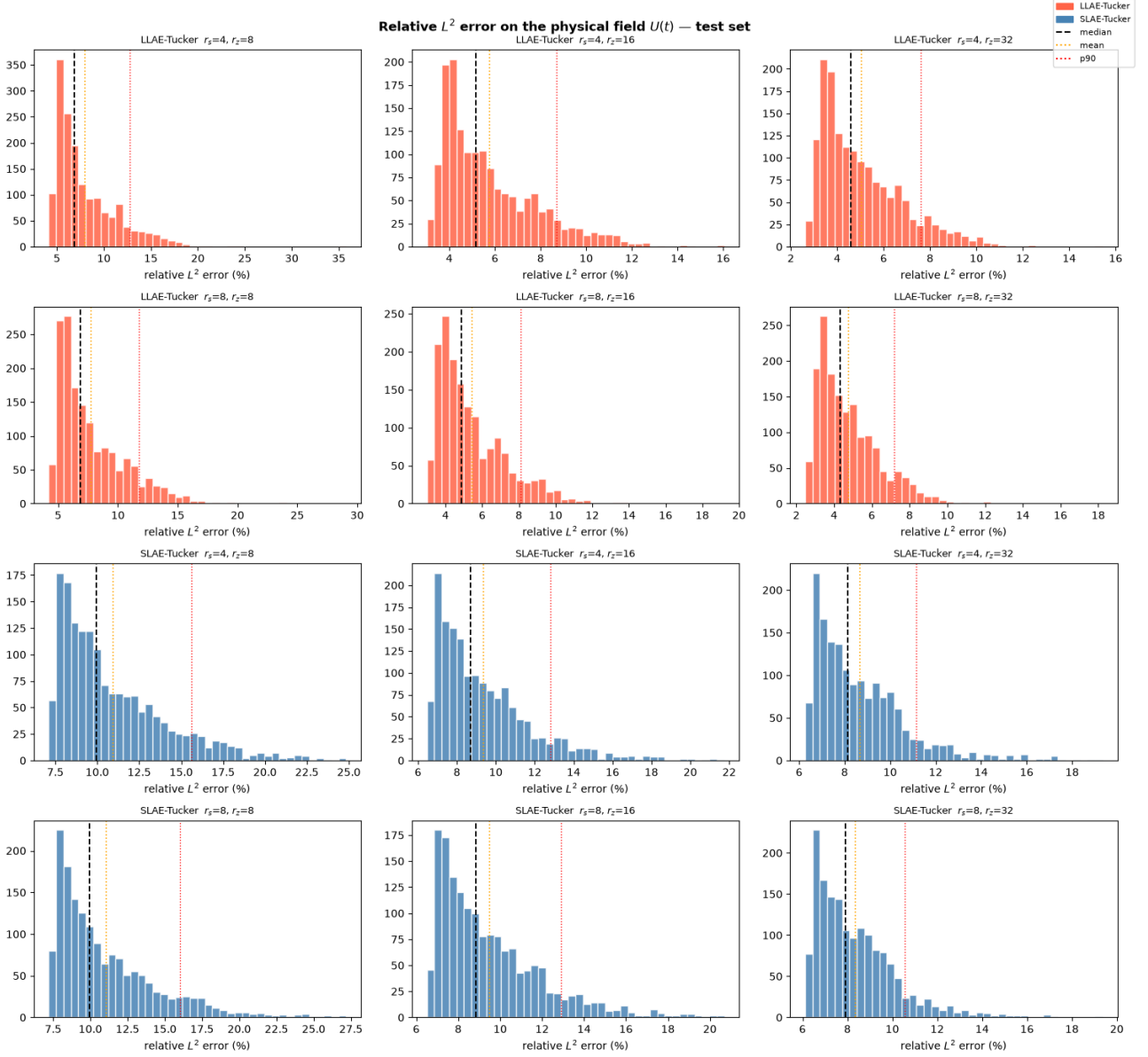


Figure 23: Per-checkpoint error histograms for all 12 Tucker surrogate models (test set, $n = 1620$). SLAE-Tucker in blue, LLAE-Tucker in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

B POD Baseline: Full Ablations

This appendix collects the complete hyperparameter sweeps behind Section 3.2, for both the pure POD reconstruction and the end-to-end POD surrogate.

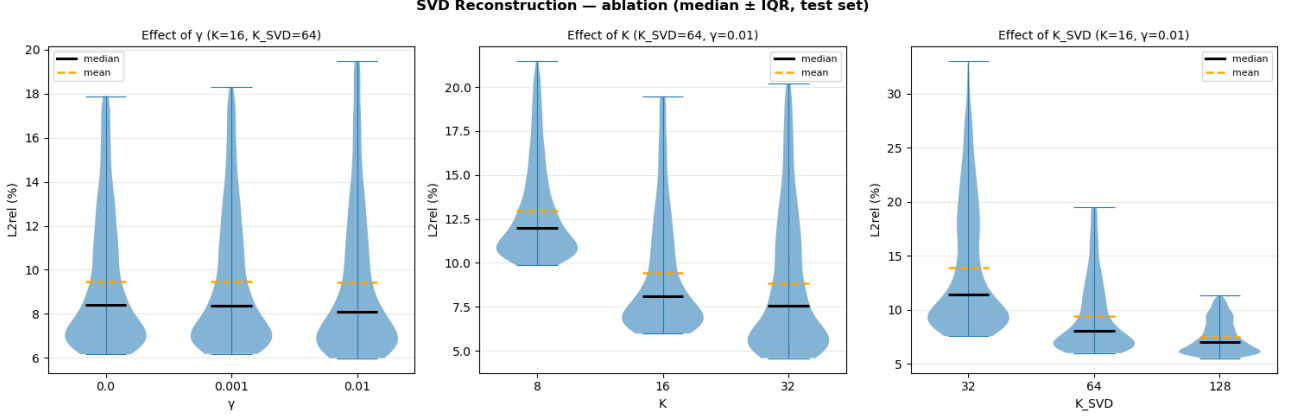


Figure 24: POD reconstruction ablation (test set, median $\pm$ IQR, violin plots; black line = median, dashed orange = mean). From left to right: effect of the damping parameter γ ($K = 16$, $k_{\text{pod}} = 64$), effect of the number of Laplace frequencies K ($k_{\text{pod}} = 64$, $\gamma = 0.01$), and effect of the number of POD modes k_{pod} ($K = 16$, $\gamma = 0.01$).

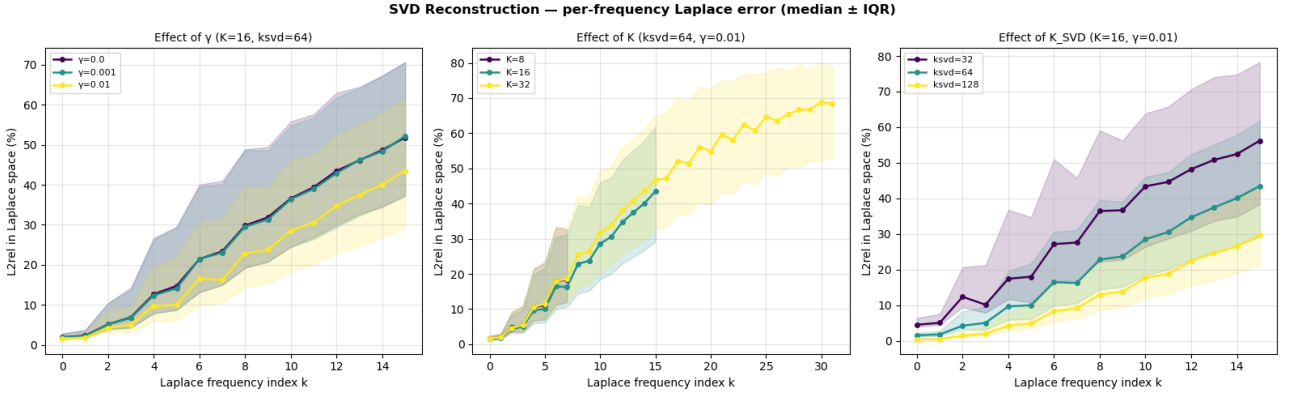


Figure 25: POD reconstruction — per-frequency Laplace-domain error (median $\pm$ IQR, test set). Left: effect of γ ; centre: effect of K ; right: effect of k_{pod} . The left panel exposes the trade-off discussed in Section 3.2: the Laplace-domain error is systematically **lower** for $\gamma = 0$ at every frequency index, yet the net physical-field error is nearly unchanged, because the Tikhonov inversion error moves in the opposite direction.

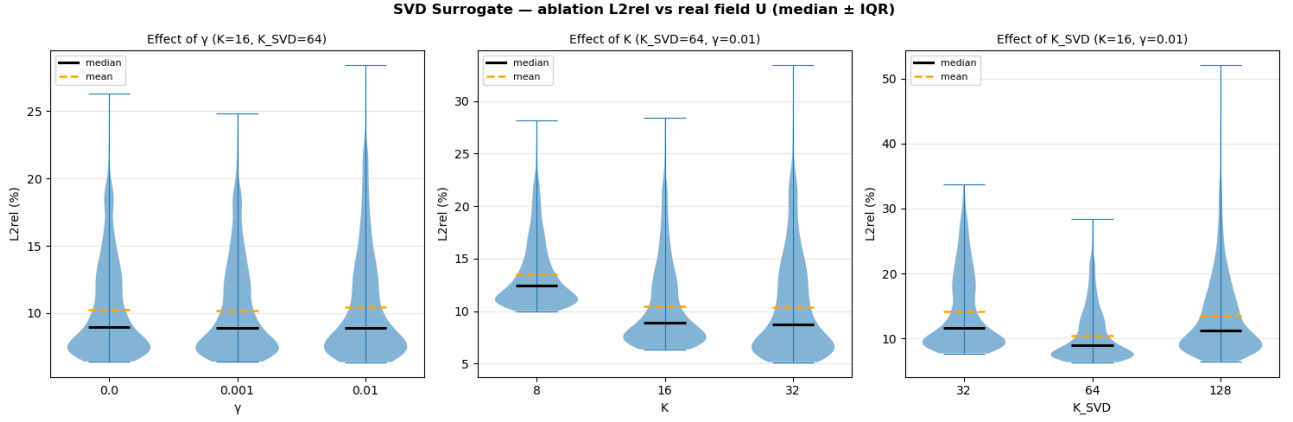


Figure 26: POD surrogate ablation (test set, median $\pm$ IQR). From left to right: effect of γ ($K = 16, k_{pod} = 64$), effect of K ($k_{pod} = 64, \gamma = 0.01$), and effect of k_{pod} ($K = 16, \gamma = 0.01$). Black line = median, dashed orange = mean.

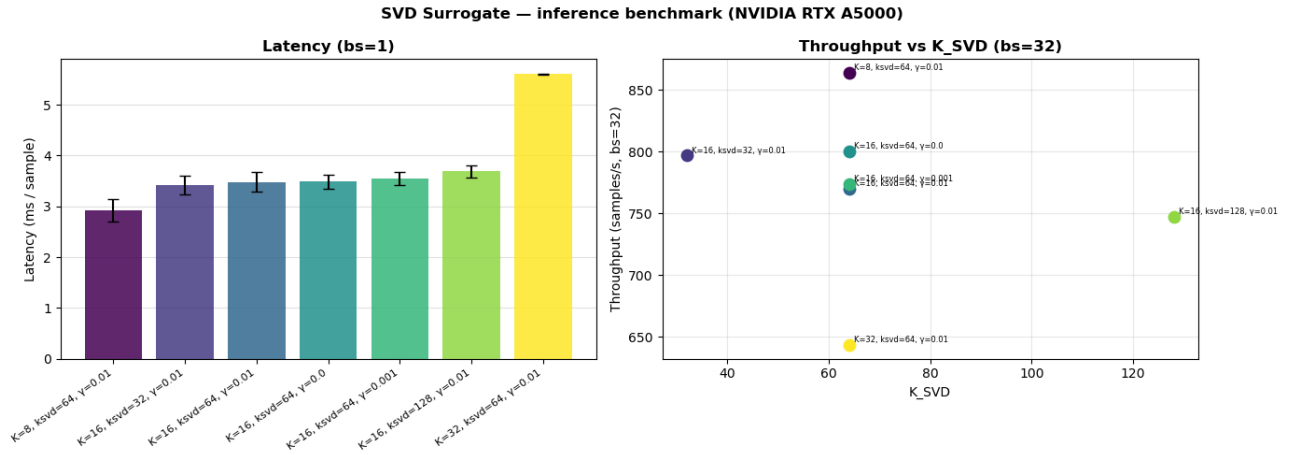


Figure 27: POD surrogate inference benchmark (NVIDIA RTX A5000, 50 timed CUDA Event runs, 10 warm-up runs). Left: per-sample latency at batch size 1, sorted by latency. Right: throughput (samples/s) vs. k_{pod} at batch size 32.

C Surrogate Ablation Studies

Figures 28 and 29 show the effect of each hyperparameter on the end-to-end surrogate error, using violin plots of the full test-set distribution (one hyperparameter varied at a time, others fixed at the baseline values of Table 9).

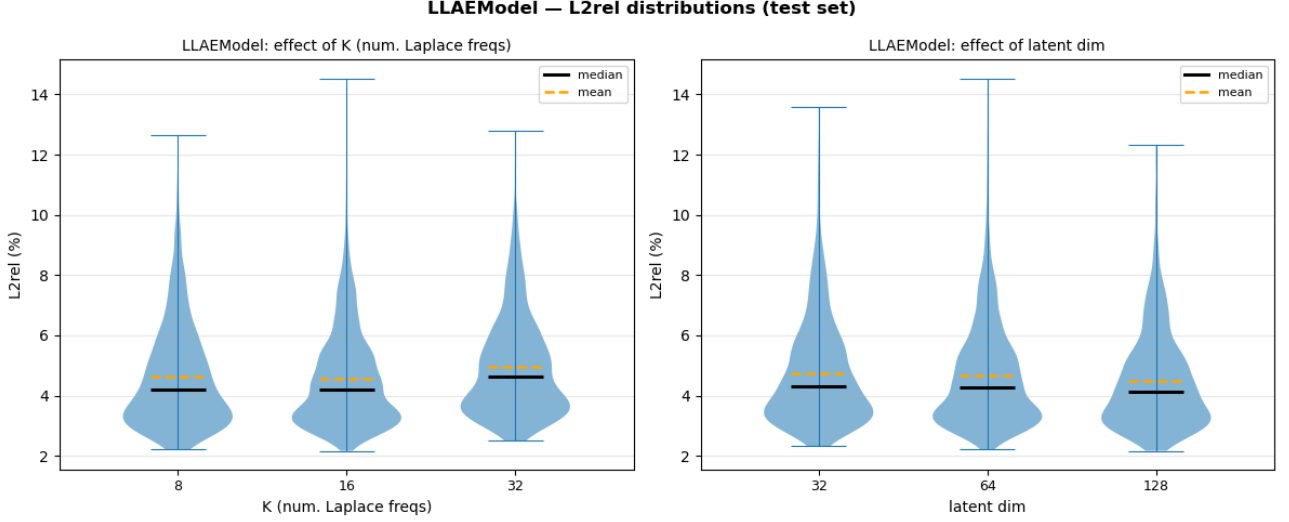


Figure 28: LLAEM surrogate ablation (test set, $n = 1620$, violin plots; black line = median, dashed orange = mean). From left to right: effect of the number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$) and effect of the latent dimension d_z ($K = 16$, $\gamma = 0.01$). The surrogate error varies by less than 0.5 pp across all LLAEM configurations.



Figure 29: SLAE surrogate ablation (test set, $n = 1620$). Same layout as Figure 28. SLAE error is strongly sensitive to K : $K = 8$ degrades to 11.6% while $K = 32$ achieves the best SLAE result (5.7%). Note the larger y-axis range compared to LLAEM.

D Governing Equations

Our neural surrogate model is designed to emulate the transient dynamics of a reacting flow system. The physical system is governed by a set of coupled partial differential equations representing the conservation of mass, momentum, energy, and chemical species.

1. Fluid Mechanics (Momentum):

$$(39) \quad \frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) - \nabla \cdot \boldsymbol{\tau} = -\nabla p + \rho \mathbf{g}$$

2. Chemical Species Transport:

$$(40) \quad \frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y_i) - \nabla \cdot (\mu_{\text{eff}} \nabla Y_i) = \dot{\omega}_{i,\text{eff}}$$

under the constraint $\sum Y_i = 1$.

3. Energy:

$$(41) \quad \frac{\partial(\rho h)}{\partial t} + \nabla \cdot (\rho \mathbf{U} h) + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\rho \mathbf{U} K) - \frac{\partial p}{\partial t} - \nabla \cdot (\alpha_{\text{eff}} \nabla h) = \dot{q}_{\text{chem}}$$

where $K = \frac{1}{2} |\mathbf{U}|^2$ is the kinetic energy.

4. Combustion Model (PaSR): The effective reaction rate is modulated by the Partially Stirred Reactor model:

$$(42) \quad \dot{\omega}_{i,\text{eff}} = \kappa \dot{\omega}_i(c^*, T^*), \quad \text{with} \quad \kappa = \frac{\tau_c}{\tau_c + \tau_{\text{mix}}}$$

where τ_c and τ_{mix} are the characteristic chemical and turbulent mixing times, respectively.

5. Equation of State: The ideal gas law closes the system:

$$(43) \quad p = \rho R T \quad \text{or} \quad \rho = \psi p \quad \text{with} \quad \psi = \frac{1}{RT}$$

E Architectural Details and Setup

All models were trained using the AdamW optimizer (weight decay 10^{-4}) with a learning rate of 5×10^{-4} for the autoencoders, 3×10^{-4} for the surrogate trunk, and 5×10^{-5} for the fine-tuned decoder, over 300 epochs. The autoencoders used a batch size of 256 (SLAE) or 16 (LLAE). Training was performed on a workstation equipped with an Intel(R) Xeon(R) w3-2435 CPU (62GB RAM) and an NVIDIA GeForce RTX 3090 GPU (24GB VRAM). Training data consists of 6480 simulations; the test set contains 1620 samples.

Table 9 summarises the hyperparameters of both autoencoder architectures.

Group	Hyperparameter	SLAE	LLAE
Architecture (baseline [†])	Latent dim d_z	64	64
	Frequencies K	16	16
	Sinusoidal levels L	8	8
	Contour γ	0.01	0.01
Regularization	Ridge β	10^{-2}	10^{-2}
	Latent roundtrip β_{latent}	—	0.1
	Tikhonov α_t	7×10^{-3} (eval)	7×10^{-3}
	Tikhonov λ	3×10^{-5} (eval)	3×10^{-5}
Training	Epochs	300	300
	Batch size	256	16
	Learning rate	5×10^{-4}	5×10^{-4}
	Early stopping patience	40	40

Table 9: Autoencoder hyperparameters. [†]Baseline values fixed when the other architectural parameters are swept: $d_z \in \{32, 64, 128\}$, $K \in \{8, 16, 32\}$, $\gamma \in \{0.0, 0.001, 0.01\}$.

Table 10 summarises the hyperparameters used for surrogate training (Phase 2).

Group	Hyperparameter	SLAE	LLAE
Architecture (fixed)	Trunk layers n_{trunk}	4	4
	Trunk hidden dim	512	512
	Head layers n_{head}	2	2
	Head hidden dim	256	256
Loss	Latent weight α_{lat}	1.0	1.0
	Spatial weight α_{spat}	1.0	1.0
	Tikhonov α_t	7×10^{-3}	7×10^{-3}
	Tikhonov λ	3×10^{-5}	3×10^{-5}
Training	Epochs	300	300
	Batch size	256	16
	$lr_{\text{surrogate}}$	3×10^{-4}	3×10^{-4}
	lr_{decoder}	5×10^{-5}	5×10^{-5}
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40

Table 10: Surrogate training hyperparameters (Phase 2). MLP architecture and training setup are fixed across all runs; one surrogate is trained per AE checkpoint (14 total, sweeping d_z , K , γ as in Table 9). The AE decoder is fine-tuned at lr_{decoder} while the surrogate MLP is trained at $lr_{\text{surrogate}}$.

F Proofs for the Bias–Variance Contour Optimization

Proposition F.1 (Invertibility of A). *Since $\lambda > 0$ and $D_t^T D_t$ is positive semi-definite, $L_{\text{reg}} = \alpha_t D_t^T D_t + \lambda I$ is positive definite ($L_{\text{reg}} \succeq \lambda I$). Thus $A = \Re(F^H F) + L_{\text{reg}}$ is symmetric positive definite and hence invertible. The unique reconstructed solution is $u^*(\mu) = A^{-1} \Re(F^H \hat{u}(\mu))$.*

Proof of Theorem 2.1. Starting from decomposition (22) and applying the triangle inequality:

$$\|u(\mu) - u_{\text{rec}}(\mu)\| \leq \|u(\mu) - u^*(\mu)\| + \|u^*(\mu) - u_{\text{rec}}(\mu)\|.$$

The first term equals $\|A^{-1} L_{\text{reg}} u(\mu)\|$ by (23). For the second term:

$$u^*(\mu) - u_{\text{rec}}(\mu) = A^{-1} \Re\left(F^H (\hat{u}(\mu) - \tilde{u}(\mu))\right).$$

Applying sub-multiplicativity of the operator norm and $\|\Re(M)\| \leq \|M\|$ yields:

$$\|u^*(\mu) - u_{\text{rec}}(\mu)\| \leq \|A^{-1} F^H\| \cdot \|\hat{u}(\mu) - \tilde{u}(\mu)\|_{\mathbb{C}^K},$$

which gives (24). $\square$

Proof ($\epsilon_{\text{AE}}(s_k) \leq \mathcal{E}_{\text{SVD}}(s_k)$). A linear autoencoder with bottleneck dimension d_z minimizing the mean squared reconstruction error recovers exactly the truncated SVD projection [1]. Since a nonlinear autoencoder with the same bottleneck dimension optimizes over a strictly larger class of functions (which includes all linear encoder–decoder pairs), its minimum reconstruction error satisfies $\epsilon_{\text{AE}}(s_k) \leq \mathcal{E}_{\text{SVD}}(s_k)$, as empirically confirmed in [30]. $\square$

References

- [1] Pierre Baldi and Kurt Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989.
- [2] Qianying Cao, Somdatta Goswami, and George Em Karniadakis. LNO: Laplace neural operator for solving differential equations, 2023.
- [3] L. De Lathauwer, B. De Moor, and J. Vandewalle. On the best rank-1 and rank- $(r_1, r_2, \dots, r_n)$ approximation of higher-order tensors. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1324–1342, 2000.
- [4] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling, 2025.
- [5] Stefania Fresca, Luca Dedè, and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. *Journal of Scientific Computing*, 87(2):61, 2021.
- [6] Stefania Fresca and Andrea Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.
- [7] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems. 2026.

- [8] Silke Glas, Antonia Mayerhofer, and Karsten Urban. Two ways to treat time in reduced basis methods. In Peter Benner, Mario Ohlberger, Anthony Patera, Gianluigi Rozza, and Karsten Urban, editors, *Model Reduction of Parametrized Systems*, volume 17 of *MS&A*, pages 113–128. Springer, 2017.
- [9] et al. Goeckner. Model order reduction in contour integral methods for parametric pdes. 2022.
- [10] Nicola Guglielmi, María López-Fernández, and Giancarlo Nino. Numerical inverse laplace transform for convection-diffusion equations. 2020.
- [11] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022.
- [12] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.
- [13] Jan S Hesthaven, Gianluigi Rozza, and Benjamin Stamm. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*. Springer, 2015.
- [14] Jan S Hesthaven and Stefano Ubbiali. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *Journal of Computational Physics*, 363:55–78, 2018.
- [15] Michael Urs Lars Kastor, Jan Rottmayer, Anna Hundertmark, and Nicolas Ralph Gauger. Parameter conditioned interpretable u-net surrogate model for data-driven predictions of convection-diffusion-reaction processes. 2026.
- [16] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [17] Toni Lassila, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Model order reduction in fluid dynamics: Challenges and perspectives. In Alfio Quarteroni and Gianluigi Rozza, editors, *Reduced Order Methods for Modeling and Computational Reduction*, volume 9 of *MS&A*, pages 235–274. Springer, 2014.
- [18] Kookjin Lee and Kevin T Carlberg. Model reduction of dynamical systems on non-linear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- [19] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.
- [20] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [21] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.
- [22] Swaroop Mallick and Monika Mittal. Ai-based model order reduction techniques: A survey. 2025.

- [23] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.
- [24] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank reduction autoencoders. 2025.
- [25] Pranshu Pant, Ruchit Doshi, Pranav Bahl, and Amir Barati Farimani. Deep learning for reduced order modelling and efficient temporal evolution of fluid simulations. *Physics of Fluids*, 33(10), 2021.
- [26] Eric Parish, Masayuki Yano, Irina Tezaur, and Traian Iliescu. Stabilization of cdr roms. 2025.
- [27] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [28] Ricardo Reyes. The frequency reduced-basis method: Reduced order models for time-dependent problems using the laplace transform. 2025.
- [29] Tian Wang and Chuang Wang. Latent Neural Operator for solving forward and inverse PDE problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [30] Yasi Wang, Hongxun Yao, and Sicheng Zhao. Auto-encoder based dimensionality reduction. *Neurocomputing*, 184:232–242, 2016.